1. Цифровое изображение: параметры, свойства и характеристики

Цифровое изображение представляет собой **дискретное представление** визуальной информации в виде двумерного массива пикселей. Каждый пиксель имеет числовое значение, которое соответствует яркости или цвету в данной точке изображения.

Представление

$$I(x, y), \quad x = 1..M,\ y = 1..N$$

Каждый элемент матрицы — яркость пикселя.

Основными параметрами цифрового изображения являются:
1) **размер,** который определяется количеством пикселей по горизонтали и вертикали,
2) **пространственное разрешение**, показывающее плотность пикселей (ppi dpi),
3) **битовая глубина**, определяющая количество возможных уровней яркости или цвета L=2^b ,
4) **цветовая модель**, например RGB или оттенки серого (задает как именно кодируется цвет).

Цифровое изображение обладает такими свойствами, как
-**дискретность**, поскольку оно ограничено по координатам и значениям яркости,
-**квантование**, то есть округление яркости до конечного числа уровней
-**избыточность** то есть в изображении много повторяющейся информации благодаря чему его можно сжимать.

К основным характеристикам изображения относят яркость, контраст (различие между темными и светлыми областями), гистограмму распределения яркостей и наличие шума, который возникает при формировании и передаче изображения.

2. Цифровое изображение: математический аппарат

Цифровое изображение рассматривается как двумерный сигнал. Оно описывается в непрерывном случае функцией l(x,y) или в цифровом случае матрицей l[m,n]. Где каждому элементу матрицы соответствует значение яркости конкретного пикселя.

Самый базовый аппарат это линейная алгебра тк изображение это матрица и над матрицами можно делать операции. Для обработки изображений важно понимать поэлементные операции и операции фильтрации. Поэлементные операции это когда новый пиксель вычисляется только из пикселей в той же позиции, например сложение двух изображений или умножение на константу. Фильтрация это когда новый пиксель зависит от соседей, и это описывается свёрткой. Свёртка это “скользящее окно” или “ядро” которое суммирует окрестность с весами.

Также применяется корреляция и статистический анализ, позволяющий оценивать среднюю яркость, дисперсию и другие параметры.



Что входит в раздел

10 Введение в математический аппарат, применяемый в цифровой обработке изображений

- линейные и нелинейные преобразования
- арифметические операции
- логические операции
- поэлементные и матричные операции
- пространственные операции
- векторные и матричные операции
- преобразования изображений
- вероятностные методы

3. Цифровое изображение: способы формирования

Цифровое изображение — это дискретная матрица яркостей, формируемая путём дискретизации и квантования, описываемая математически и обрабатываемая с помощью арифметических и логических операций.

Процесс начинается с физического сигнала чаще всего света. В цифровой камере свет попадает на матрицу CCD или CMOS. Каждый элемент матрицы превращает свет в электрический заряд, потом сигнал усиливают, дальше через АЦП переводят в цифровой код и получают яркость пикселя. В цветной камере добавляются цветовые фильтры и дальше происходит восстановление полного RGB по данным матрицы.

Другой способ это сканирование. Сканер “пробегает” по изображению и измеряет отражённый свет строка за строкой, затем опять же оцифровывает сигнал и получается матрица пикселей. Ещё способ это синтез изображений когда картинка создаётся программно например в компьютерной графике или при моделировании, тогда численные значения пикселей генерируются по модели сцены. Есть и специальные способы формирования в медицине или технике, например КТ, МРТ, ультразвук, спутниковые снимки, где сначала измеряется физический параметр, а потом он переводится в изображение.

Независимо от способа формирования, изображение проходит этапы **приёма сигнала, его преобразования, дискретизации, квантования и кодирования**.

4. Цифровое изображение: дискретизация и квантование

Цифровое изображение — это дискретная матрица яркостей, формируемая путём дискретизации и квантования, описываемая математически и обрабатываемая с помощью арифметических и логических операций.

Дискретизация и квантование являются ключевыми этапами получения цифрового изображения из непрерывного.

Дискретизация заключается в разбиении изображения по пространственным координатам на отдельные элементы, то есть пиксели. От шага дискретизации зависит пространственное разрешение изображения и его детализация.

Переход от непрерывного изображения к дискретному:

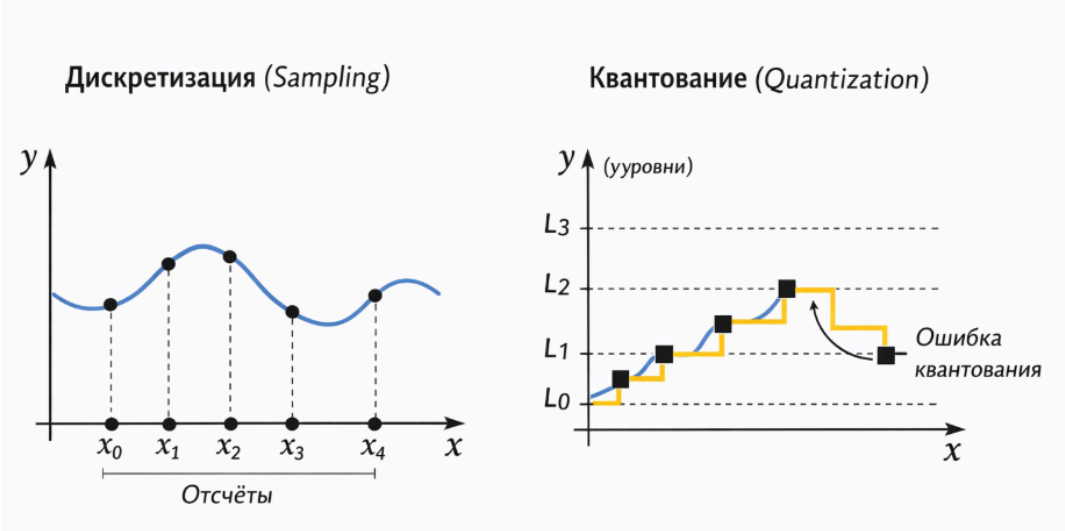
$$I[m,n] = I(x,y)\big|_{x=m\Delta x,\ y=n\Delta y}$$

Квантование представляет собой процесс округления значений яркости до конечного числа уровней, которое определяется битовой глубиной. Чем меньше уровней квантования, тем сильнее проявляются искажения в виде квантового шума.

5. Формула квантования

$$I_q[m,n] = Q\{I[m,n]\}$$

где $Q\{\cdot\}$ — оператор квантования.



5. Цифровое изображение: логические и арифметические операции

Цифровое изображение — это дискретная матрица яркостей, формируемая путём дискретизации и квантования, описываемая математически и обрабатываемая с помощью арифметических и логических операций.

Арифметические операции, такие как сложение, вычитание, умножение и деление, применяются для изменения яркости, контраста и для сравнения изображений. Например, вычитание изображений позволяет обнаруживать изменения между кадрами.



Увеличение яркости изображения на константу

10 Введение в математический аппарат, применяемый в цифровой обработке изображений

$$z(i,j) = x(i,j) + c$$

осветление/затемнение изображения



Сложение либо вычитание равных по размеру изображений

10 Введение в математический аппарат, применяемый в цифровой обработке изображений

$$z(i,j) = \alpha x(i,j) \pm (1-\alpha)y(i,j),$$

где

- x, y, z - цифровые изображения одного размера,
- $0 < \alpha < 1$ - константа.

Произведение двух изображений

1О Введение в математический аппарат, применяемый в цифровой обработке изображений

- поэлементное произведение двух изображений
  - принятые обозначения: $\odot$, $\otimes$
  - формула:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \odot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11} \cdot b_{11} & a_{12} \cdot b_{12} \\ a_{21} \cdot b_{21} & a_{22} \cdot b_{22} \end{bmatrix}$$

- матричное произведение изображений:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11} \cdot b_{11} + a_{12} \cdot b_{21} & a_{11} \cdot b_{12} + a_{12} \cdot b_{22} \\ a_{21} \cdot b_{11} + a_{22} \cdot b_{21} & a_{21} \cdot b_{12} + a_{22} \cdot b_{22} \end{bmatrix}$$

- если не сказано иного, предполагается **поэлементное** умножение

Логические операции используются в основном для бинарных изображений и масок. AND используется чтобы оставить объект только там где маска равна 1, OR объединяет области, XOR показывает различия, NOT инвертирует изображение.

6. Геометрические линейные и нелинейные преобразования

Геометрические преобразования заключаются в пространственном изменении расположения совокупности пикселей из одной двухмерной системы координат в другую.
При выполнении геометрических преобразований исходное множество пикселей с целочисленными координатами (x, y) преобразуется в новое множество (сетка растра трансформируется пространственно) с координатами (x', y') и сохранением яркости в этих пикселах.

Линейные геометрические преобразования характеризуются тем, что функции f и g являются линейными по (x, y) с возможным добавлением свободных членов. Наиболее важный для обработки изображений подкласс линейных преобразований составляют аффинные

Нелинейные геометрические преобразования определяются тем, что хотя бы одна из функций f или g является нелинейной по координатам. В результате такие преобразования, как правило, не сохраняют прямые линии и могут переводить их в кривые. Нелинейные преобразования широко применяются для моделирования и компенсации оптических искажений, например радиальной и тангенциальной дисторсии объективов, а также для построения деформаций в задачах регистрации и морфинга.

7. Аффинные преобразования

Аффинные преобразования это основной класс линейных геометрических преобразований в задачах обработки изображений. Они включают сдвиг, масштабирование, поворот, отражение и
В целом аффинные преобразования на плоскости описываются уравнениями:

$$x' = ax + bx + c$$
$$y' = dy + ey + f$$

- *Сдвиг изображения* задается соотношениями:

$$x' = x + c, y' = y + f$$

$$T_{cd} = \begin{bmatrix} 1 & 0 & c \\ 0 & 1 & f \\ 0 & 0 & 1 \end{bmatrix}$$

- *Отражение изображения относительно оси OY*

$$x' = x, y' = -y$$

$$T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- *Масштабирование (растяжение) изображения*

$$x' = \alpha x, \alpha > 0$$
$$y' = \beta y, \beta > 0$$

$$T_{ma} = \begin{bmatrix} \alpha & 0 & 0 \\ 0 & \beta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- *Скос изображения вдоль оси OX*

$$x' = x + sy$$
$$y' = y$$

$$T_{ck} = \begin{bmatrix} 1 & s & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- *Поворот изображения* вокруг начала координат на угол ϕ описывается формулами

$$x' = x\cos\phi - y\sin\phi$$
$$y' = x\sin\phi + y\cos\phi$$

$$T_{ck} = \begin{bmatrix} \cos\phi & -\sin\phi & 0 \\ \sin\phi & \cos\phi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- прямая линия переходит в прямую
- параллельные линии переходят в параллельные
- сохраняются отношения длин отрезков, лежащих на одной прямой или параллельных прямых
- сохраняется отношение площадей фигур

8. Проективные преобразования

Проективные преобразования занимают промежуточное место между аффинными и произвольными нелинейными отображениями и описывают отображения плоскости, возникающие при перспективной проекции, то есть при наблюдении плоской сцены камерой. Они задаются гомографией и в координатной форме записываются как $x' = \frac{h_{11}x+h_{12}y+h_{13}}{h_{31}x+h_{32}y+h_{33}}$ и $y' = \frac{h_{21}x+h_{22}y+h_{23}}{h_{31}x+h_{32}y+h_{33}}$, где коэффициенты h_{ij} определяют матрицу $\mathbf{H}$ размера 3×3 с точностью до ненулевого множителя. Проективные преобразования сохраняют прямые линии, однако не сохраняют параллельность, что отражает наличие точек схода в перспективе. Именно поэтому они используются для коррекции перспективных искажений, выпрямления документов, построения панорам и перехода к “вид сверху” при известной плоскости сцены.

9. Некоторые фундаментальные отношения между пикселями: меры расстояний

В дискретной модели изображения пиксели рассматриваются как точки решётки $\mathbb{Z}^2$, и для двух пикселей $p = (x, y)$ и $q = (s, t)$ вводят меры расстояния, которые задают разные геометрии и используются в анализе изображений. Наиболее распространёнными являются евклидова метрика

$$D_E(p, q) = \sqrt{(x - s)^2 + (y - t)^2},$$

городская метрика

$$D_4(p, q) = |x - s| + |y - t|,$$

и шахматная метрика

$$D_8(p, q) = \max\{|x - s|,\ |y - t|\}.$$

10. Некоторые фундаментальные отношения между пикселями: смежность, связность, области и границы

Смежность — это отношение между пикселями, определяющее, считаются ли они соседними (то есть могут ли быть соединены напрямую).
Для пикселя $p = (x, y)$ вводятся следующие типы окрестностей:

$$N_4(p) = \{(x \pm 1, y), (x, y \pm 1)\}, \quad N_D(p) = \{(x \pm 1, y \pm 1)\}, \quad N_8(p) = N_4(p) \cup N_D(p)$$

Пиксели p и q называются **4-смежными**, если $q \in N_4(p)$;
8-смежными, если $q \in N_8(p)$.
Выбор типа смежности определяет топологию объекта: при 4-смежности пиксели соединяются только по сторонам, при 8-смежности — и по диагонали.

Иногда используется **m-смежность (смешанная)**, чтобы избежать неоднозначностей при касании диагоналей.
Два пикселя p и q считаются m-смежными, если:

- $q \in N_4(p)$, или
- $q \in N_D(p)$ и при этом $N_4(p) \cap N_4(q) = \varnothing$.

Связность множества пикселей $R \subset \mathbb{Z}^2$ определяется существованием пути $p = p_0, p_1, \ldots, p_k = q$, где каждый следующий пиксель смежен предыдущему и все $p_i \in R$.
Если такой путь существует, говорят, что пиксели p и q связаны.
Таким образом, связность описывает возможность пройти от одного пикселя к другому через цепочку смежных пикселей внутри области.

Область — это связное множество пикселей, принадлежащих одному объекту или имеющих одинаковое значение интенсивности.
На бинарном изображении каждая область соответствует одному “объекту” (связной компоненте).

Граница области — это множество пикселей области, имеющих хотя бы одного соседа, не принадлежащего этой области.

11. Некоторые фундаментальные отношения между пикселями: соседи отдельного элемента

Локальная структура изображения описывается окрестностями пикселя. Для $p = (x, y)$ вводят 4-окрестность

$$N_4(p) = \{(x \pm 1, y), (x, y \pm 1)\},$$

диагональную окрестность

$$N_D(p) = \{(x \pm 1, y \pm 1)\},$$

и 8-окрестность

$$N_8(p) = N_4(p) \cup N_D(p).$$

На основе окрестностей формализуется смежность: пиксели 4-смежны тогда и только тогда, когда $q \in N_4(p)$, а 8-смежны тогда и только тогда, когда $q \in N_8(p)$.

12. Основные стадии цифровой обработки изображений

Сначала выполняют формирование изображения, то есть получение данных с датчика и перевод в цифровой вид с дискретизацией и квантованием, а также базовой калибровкой. Затем следует предварительная обработка, цель которой — подавление шума, нормализация яркости и контраста, геометрическая коррекция и приведение изображения к удобному виду. Далее выполняют сегментацию, то есть разбиение изображения на области или выделение объекта и фона с формированием маски или карты меток. После сегментации проводят описание объектов, вычисляя признаки формы, текстуры и статистики яркости, чтобы заменить пиксели компактным информативным представлением. На следующей стадии выполняют распознавание и интерпретацию, то есть классификацию, обнаружение или оценку параметров по признакам. Завершают постобработкой и визуализацией, где уточняют результат и представляют его пользователю или системе.

13. Линейные и кусочно-линейные преобразования яркости полутоновых изображений: препарирование изображений, бинаризация, линейное растяжение контраста изображения. Преимущества и недостатки

Линейные и кусочно-линейные преобразования яркости — это **точечные операции** над полутоновым изображением: каждый пиксель меняется по правилу $s = T(r)$, где r — исходная яркость, s — новая. Линейное преобразование имеет вид $s = ar + b$, а кусочно-линейное задаётся несколькими линейными участками на разных диапазонах яркости.

Препарирование — это выделение нужного диапазона яркостей и подавление остальных (например, усиливаем контраст в интервале, а вне интервала “сжимаем” или обрезаем). Суть в том, что мы делаем интересующие объекты заметнее за счёт изменения наклона функции на выбранном участке. Плюсы: очень просто и быстро, хорошо подчёркивает нужные детали. Минусы: требует правильного выбора диапазона, легко потерять информацию вне него, может усиливать шум.

Бинаризация — это преобразование полутонового изображения в бинарное, где каждый пиксель принимает одно из двух значений, обычно 0 и $L-1$ (или 0 и 1). Наиболее распространённый вариант — пороговая бинаризация: выбирают порог T и задают правило $s=0$, если $r<T$, и $s=L-1$, если $r \geq T$. Бинаризация является базовой операцией сегментации, потому что позволяет отделить объект от фона при достаточно раздельных распределениях яркости. Основные преимущества бинаризации — максимальная простота, высокая скорость и резкое упрощение дальнейшего анализа (поиск связных компонент, контуров, морфология). Недостатки — высокая чувствительность к шуму, неоднородному освещению и неверному выбору порога; при слабом контрасте объект и фон могут сливаться, а мелкие детали теряться. Поэтому на практике часто

Линейное растяжение контраста — когда изображение занимает лишь часть диапазона яркостей, его растягивают так, чтобы минимальная яркость стала 0, а максимальная — $L-1$, а всё между ними масштабировалось линейно. Плюсы: простое улучшение “тусклого” изображения, повышает общий контраст. Минусы: чувствительно к выбросам (один очень яркий/тёмный пиксель портит растяжение), усиливает шум и не решает проблемы локального освещения.

---

14. Нелинейные преобразования яркости изображений: гамма-коррекция, логарифмическая коррекция, соляризация изображения. Преимущества и недостатки

1) Гамма-коррекция

Суть. Гамма-коррекция — степенное преобразование, которое **перераспределяет яркости неравномерно**, в основном меняя **средние тона** и тени. Используется для коррекции отображения (экраны/камеры) и “вытягивания” теней или, наоборот, приглушения светов.

Формула (обычно в нормированном виде $r \in [0, 1]$).

$$s = r^{\gamma}$$

Иногда пишут $s = r^{1/\gamma}$ — это зависит от соглашения (что называют “гаммой” в конкретном курсе).

Эффект.

- $\gamma < 1$ → изображение **светлеет**, лучше видны детали в тенях.
- $\gamma > 1$ → изображение **темнеет**, “прижимаются” яркие области.

Плюсы. Простая, быстрая, хорошо управляет яркостью средних тонов, используется как стандартная коррекция отображения.
Минусы. Нужен подбор γ, может **усиливать шум в тенях** (при $\gamma < 1$), глобальная операция — плохо справляется с неравномерным освещением.

---

2) Логарифмическая коррекция

Суть. Логарифмическое преобразование выполняет **сжатие динамического диапазона**: очень яркие значения “сжимает”, а тёмные — **растягивает**. Это полезно, когда есть огромный разброс яркостей или нужно “вытащить” детали в тенях.

$$I = clog(I + b)$$

где

- c - константа
- $b \leq 1$

Цель - сжатие динамического диапазона при визуализации данных.

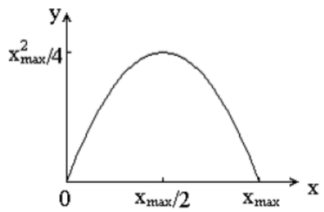
Плюсы. Эффективно работает при большом динамическом диапазоне, вытягивает детали в тёмных областях.
Минусы. Может **снижать контраст в светах**, тоже усиливает шум в тенях, требует настройки c, глобальная операция.

Соляризация - изменяет яркость с помощью параболической функции

$$y = kx(x_{max} - x)$$

где

- k - константа, управляющая динамическим диапазоном преобразованного изображения
- x_{max} - максимальное значение яркости исходного изображения



. Функция изменения яркости при соляризации

Участки исходного изображения с яркостью, близкой к белому цвету, после обработки принимают значения яркости ближе к черному. При этом темные участки остаются темными.

Плюсы. Может подчёркивать границы и создавать сильный визуальный эффект, иногда полезна для визуализации.

Минусы. Сильно искажает фотометрический смысл яркости, обычно непригодна для “улучшения качества” в инженерном смысле, больше эффект/спецобработка.

---

15. Преобразования яркости на основе гистограммы. Эквализация

Гистограммой цифрового изображения с уровнями яркости в диапазоне $[0, L-1]$ называется дискретная функция $h(r_k) = n_k$, где r_k есть k-й уровень яркости, а n_k — число пикселей на изображении, имеющих яркость r_k
Нормализованная гистограмма:

$$p(r_k) = \frac{h(r_k)}{MN} = \frac{n_k}{MN}$$

Эквализация гистограммы — это глобальное преобразование яркости, которое повышает контраст за счёт перераспределения уровней яркости по всему диапазону. Сначала по исходному изображению $f(x, y)$ строят гистограмму $H(j)$ и считают число пикселей N. Затем гистограмму нормируют, получая оценку вероятностей $p(j) = H(j)/N$, и вычисляют кумулятивную функцию распределения $F(i) = \sum_{j=0}^{i} p(j)$. После этого задают отображение уровней яркости $T(i) = \lfloor (L-1)F(i) \rfloor$ и применяют его к каждому пикселю, то есть $g(x, y) = T(f(x, y))$. В результате часто встречающиеся уровни “растягиваются” по диапазону, изображение использует больше уровней яркости и выглядит более контрастным, но метод глобальный, поэтому может усиливать

---

16. Низкочастотная фильтрация в пространственной области (арифметический и геометрический усредняющие фильтры, контргармонический усредняющий фильтр, фильтр Гаусса)

Низкочастотная фильтрация в пространственной области — это сглаживание изображения за счёт усреднения значений в локальной окрестности. Цель таких фильтров — подавление высокочастотных компонент (шум, мелкие детали, резкие границы). Обычно результат в точке (x, y) получают, применяя к окну S_{xy} вокруг пикселя некоторую агрегирующую функцию.

Арифметический усредняющий фильтр, или *box-фильтр*, усредняет значение яркости пиксела по окрестности путем использования маски с одинаковыми коэффициентами:

$$f(x,y) = \frac{1}{M \cdot N} \sum_{i=0}^{M} \sum_{j=0}^{N} g(i,j)$$

где

- $f(x, y)$ - значение пиксела выходного изображения;
- $g(i,j)$ - текущее значение пиксела исходного изображения, соответствующее центру маски;
- M и N - ширина и высота маски соответственно.



Геометрический усредняющий фильтр
7 Преобразования изображений локального типа

$$f(x,y) = \left[\prod_{i=0}^{M} \prod_{j=0}^{N} g(i,j) \right]^{\frac{1}{M \cdot N}}.$$

При таком усреднении происходит сглаживание изображения, аналогичное арифметическому усреднению, наблюдается ухудшение резкости, свойственное всем фильтрам из этого класса, но отдельные объекты исходного изображения искажаются меньше.



Гармонический усредняющий фильтр
7 Преобразования изображений локального типа

$$f(x,y) = \frac{M \cdot N}{\sum_{i=0}^{M} \sum_{j=0}^{N} \frac{1}{g(i,j)}}$$

Данный фильтр хорошо работает с импульсными шумами типа «соль» (белыми точками) и не работает с шумами типа «перец».

Контргармонический усредняющий фильтр

7 Преобразования изображений локального типа

$$f(x,y) = \frac{\sum_{i=0}^{M}\sum_{j=0}^{N} g(i,j)^{Q+1}}{M \cdot N \cdot \sum_{i=0}^{M}\sum_{j=0}^{N} g(i,j)^{Q}}$$

где Q - порядок фильтра. Контргармонический фильтр идеально подходит для уменьшения или полного устранения импульсных шумов типа «соль-и-перец», причем при положительных значениях Q подавляются шумы «перец», а при отрицательных - «соль». Однако одновременное удаление биполярного

Фильтр Гаусса

7 Преобразования изображений локального типа

Пикселы, расположенные ближе к анализируемому пикселу, оказывают большее влияние на яркость, вычисляемую в процессе фильтрации.
К такому типу фильтров относятся фильтры, значения весов которых описываются колоколообразной функцией Гаусса

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

где

- μ - координата центральной точки;
- σ - вещественная константа, определяющая ширину колокола.

---

17. Высокочастотная фильтрация в пространственной области (фильтр Собеля, Превитта, Лапласа)

Высокочастотный фильтр – ослабляет низкочастотные компоненты в изображении и усиливает роль высокочастотных. Фильтры высокой частоты применяются для выделения таких деталей, как контуры, границы или для повышения резкости изображения. Каждый скачок яркости и каждый контур

Фильтр Превитта (Prewitt)

Превитт аппроксимирует частные производные по x и y с помощью двух 3×3 масок.

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}, \qquad G_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

После свёртки получают приближения g_x и g_y, а величину градиента часто оценивают как

$$|G| = \sqrt{g_x^2 + g_y^2} \quad \text{или} \quad |G| \approx |g_x| + |g_y|.$$

Фильтр Собеля (Sobel)

Собель также вычисляет приближения производных, но придаёт больший вес центральной строке/столбцу, то есть сочетает дифференцирование и лёгкое сглаживание.

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Далее так же находят g_x, g_y и модуль градиента:

$$|G| = \sqrt{g_x^2 + g_y^2} \quad \text{или} \quad |G| \approx |g_x| + |g_y|.$$

Иногда также используют направление градиента:

$$\theta = \arctan\left(\frac{g_y}{g_x}\right).$$

Фильтр Лапласа (Laplacian)

Лапласовский фильтр основан на второй производной и выделяет области быстрого изменения яркости независимо от направления. Непрерывный оператор:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}.$$

В дискретном виде применяют маски 3×3, например вариант по 4-соседям:

$$H = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

или вариант по 8-соседям:

$$H = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}.$$

18. Высокочастотная фильтрация в пространственной области (алгоритм Кэнни)

Алгоритм Кэнни — классический метод выделения границ, который строит тонкие и устойчивые контуры по градиенту с подавлением шумов и двойным порогом. Основные шаги такие.

Сначала выполняют сглаживание изображения гауссовым фильтром, чтобы уменьшить влияние шума: $f_s = f * G_\sigma$. Затем вычисляют градиент сглаженного изображения, то есть приближения производных по x и y, например через Собеля: получают g_x и g_y. По ним находят модуль градиента $M = \sqrt{g_x^2 + g_y^2}$ и направление $\theta = \arctan(g_y / g_x)$.

Далее выполняют подавление немаксимумов. На этом шаге граница “утончается”: для каждого пикселя проверяют, является ли его значение M локальным максимумом вдоль направления градиента θ. Если не является, его подавляют (приравнивают к нулю), оставляя только тонкие гребни границ толщиной примерно 1 пиксель.

После этого применяют двойное пороговое значение. Задают высокий порог T_H и низкий T_L. Пиксели с $M \geq T_H$ считают сильными границами, пиксели с $T_L \leq M < T_H$ — слабыми кандидатами, а ниже T_L — отбрасывают.

Завершающий шаг — трассировка границ по гистерезису. Слабые пиксели оставляют только в том случае, если они связаны (обычно по 8-соседству) с сильными пикселями границы. Это позволяет сохранить непрерывные контуры и одновременно отбросить случайные шумовые отклики.

---

19. Внесение эффектов на изображении на примере акварелизации

Акварелизация в цифровой обработке изображений — это имитация акварельной живописи, которая достигается тем, что фотографию преобразуют в сочетание крупных “заливок” и мягко подчёркнутых контуров. Для этого обычно сначала делают сглаживание с сохранением границ (например bilateral или guided), чтобы убрать мелкую фактуру и оставить крупные однородные области, затем уменьшают число оттенков (квантование/постеризация), чтобы цвета стали более “рисовальными”, параллельно выделяют контуры (градиентные фильтры или Кэнни) и после порога получают линийную маску, а затем накладывают эти контуры на сглаженное изображение; иногда дополнительно добавляют текстуру бумаги и лёгкую тоновую коррекцию, чтобы результат выглядел ближе к настоящей акварели.

---

20. Медианная фильтрация

Медианная фильтрация. Помехи в виде белых или черных точек на изображении - это шумы импульсного типа. Линейные фильтры не устраняют их полностью, а лишь локально усредняют их значения. Шумы такого типа удаляются с помощью нелинейных фильтров, например медианных.

Медианная фильтрация — нелинейный метод пространственной фильтрации, относящийся к ранговым (order-statistic) фильтрам, в котором выходное значение пикселя определяется медианой выборки интенсивностей в локальной окрестности. Для окна S_{xy} вокруг точки (x, y) преобразование задаётся как

$$g(x,y) = \text{median}\{\, f(s,t) \mid (s,t) \in S_{xy} \,\}.$$

Оператор медианы минимизирует сумму модулей отклонений (L1-критерий), поэтому обладает высокой устойчивостью к выбросам и эффективно подавляет импульсные помехи (“salt-and-pepper”), сохраняя при этом резкие перепады интенсивности лучше, чем линейные усредняющие фильтры. Степень сглаживания и пространственная селективность определяются размером и формой окна; увеличение окна повышает подавление шума, но усиливает деградацию мелких структур и тонких деталей. Для многоканальных изображений медианную фильтрацию реализуют поканально либо через векторную медиану в выбранном цветовом пространстве.

21. Двумерное дискретное преобразование Фурье. Свойства

Пусть $f(x,y)$ — изображение размера $M \times N$, где $x = 0, \dots, M-1$, $y = 0, \dots, N-1$. Его 2D DFT определяется как

$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y)\, e^{-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}, \quad u = 0, \dots, M-1,\ v = 0, \dots, N-1.$$

Обратное преобразование:

$$f(x,y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u,v)\, e^{+j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}.$$

- $\frac{1}{MN}$ - положение множителя не имеет значения.
- **Базисными функциями преобразования являются экспоненты с комплексными показателями**, которые можно разложить на синусную и косинусную составляющие.
- **Значение Фурье-преобразования в начале координат равно среднему значению яркости изображения (постоянная составляющая спектра)**.
- Фурье-спектр симметричен.

- Фурье-спектр

$$|F(u,v)| = \sqrt{R^2(u,v) + I^2(u,v)}$$

- Фаза

$$\phi(u,v) = arctg[\frac{I(u,v)}{R(u,v)}]$$

- Энергетический спектр

$$P(u,v) = |F(u,v)|^2 = R^2(u,v) + I^2(u,v)$$

22. Фильтрация изображений в частотной области: формула, типы фильтров, примеры использования

- **Алгоритмы фильтрации в частотной области основываются на теореме о свертке**
- В двумерном случае преобразование свертки выглядит следующим образом:

$$G(u,v) = H(u,v)F(u,v)$$

- где
  - G - Фурье-образ результата свертки,
  - H - Фурье-образ фильтра,
  - F - Фурье-образ исходного изображения.
- То есть **в частотной области двумерная свертка заменяется поэлементным перемножением образов исходного изображения и соответствующего фильтра**.

Процедура фильтрации в частотной области состоит из следующих шагов:

- Исходное изображение умножается на $(-1)^{x+y}$, чтобы его Фурье-преобразование оказалось центрированным.
- Вычисляем прямое ДПФ $F(u,v)$ изображения, полученного после шага 1.
- Функция $F(u,v)$ умножается на функцию фильтра $H(u,v)$.
- Вычисляется обратное ДПФ от шага 3.
- Выделяется вещественная часть результата шага 4.
- Результат шага 5 умножается на $(-1)^{x+y}$

Низкочастотные фильтры (LPF) пропускают низкие частоты и подавляют высокие, поэтому дают сглаживание и подавление шума. Их применяют для уменьшения зернистости, подавления гауссового шума, устранения мелких деталей перед сегментацией. Типичный пример — гауссов НЧ-фильтр в спектре.

Высокочастотные фильтры (HPF) пропускают высокие частоты и подавляют низкие, поэтому выделяют резкие переходы яркости и повышают резкость. Их используют для подчёркивания границ, выделения контуров, повышения детализации, а также как основу “sharpening/high-boost” (усиление деталей на фоне исходного).

23. Фильтрация изображений в частотной области: высокочастотные фильтры

Высокочастотная фильтрация в частотной области применяется для подавления низких частот (фона, медленных изменений яркости) и выделения высоких (границ, мелких деталей). Делается по общей схеме: вычисляют спектр изображения $F(u,v)$, умножают на передаточную функцию $H(u,v)$ и выполняют обратное преобразование:

$$G(u,v) = H(u,v)\,F(u,v), \qquad g(x,y) = \mathcal{F}^{-1}\{G(u,v)\}.$$

- **Идеальный фильтр высоких частот** получается путем инверсии идеального низкочастотного фильтра:

$$H(u,v) = 1 - H(u,v)$$

- **Высокочастотный гауссовский фильтр** имеет вид:

$$h(x) = \sqrt{2\pi}\sigma_A A e^{-2(\pi\sigma_A x)^2} - \sqrt{2\pi}\sigma_B B e^{-2(\pi\sigma_B x)^2} \text{ и } H(u) = Ae^{-\frac{u^2}{2\sigma_A^2}} - Be^{-\frac{u^2}{2\sigma_B^2}}$$

24. Фильтрация изображений в частотной области: низкочастотные фильтры

Низкочастотная фильтрация в частотной области используется для **сглаживания** изображения и подавления **высокочастотных** компонент (мелких деталей, шума, резких границ). Общая схема такая: находят спектр изображения $F(u,v)$, умножают на передаточную функцию низкочастотного фильтра $H(u,v)$ и делают обратное преобразование:

$$G(u,v) = H(u,v)\,F(u,v), \qquad g(x,y) = \mathcal{F}^{-1}\{G(u,v)\}.$$

Здесь $H(u,v)$ близка к 1 вблизи центра спектра (низкие частоты) и стремится к 0 на периферии (высокие частоты).

- **Идеальный фильтр низких частот** $H(u,v)$ имеет вид
$$H(u,v) = 1, \text{ если } D(u,v) < D0,\ H(u,v) = 0, \text{ если } D(u,v) \geq D0,$$
- где $D(u,v) = \sqrt{(u - \frac{M}{2})^2 + (v - \frac{N}{2})^2}$ - расстояние от центра частотной плоскости.

- **Низкочастотный гауссовский фильтр** имеет вид:
$$h(x) = \sqrt{2\pi}\sigma A e^{-2(\pi\sigma x)^2} \text{ и } H(u) = Ae^{-\frac{u^2}{2\sigma^2}}$$
- Чем уже профиль фильтра в частотной области (чем больше σ), тем он шире в пространственной.

- **Низкочастотный гауссовский фильтр** в двумерном случае имеет вид:
$$H(u,v) = e^{-\frac{D^2(u,v)}{2D_0^2}}$$

25. Понятие “цвет”

Результат восприятия излучения (или отражённого света) зрительной системой при заданных условиях освещения и наблюдения. Физически мы имеем спектральное распределение света по длинам волн, а психофизиологически мозг сводит это к ощущению цвета.

Цветовое зрение обеспечивают колбочки (несколько типов с разной спектральной чувствительностью), поэтому одинаковое ощущение цвета могут давать разные спектры излучения (метамеризм). Палочки в основном отвечают за светочувствительность и не дают цветоразличения, поэтому при слабом освещении цвет воспринимается хуже; в фовеа колбочек больше, поэтому в центре поля зрения цвет и детализация различаются лучше.

С научной точки зрения удобнее разделять три уровня:

1) физический: свет характеризуется спектральной плотностью мощности, то есть тем, сколько энергии приходится на каждую длину волны.

2) физиологический: в сетчатке есть три типа колбочек, по-разному чувствительных к диапазонам длин волн, и их отклики образуют три сигнала.

3) психофизический: из этих сигналов формируется ощущение, которое мы описываем терминами вроде “красный”, “зелёный”.

26. Модели цветного изображения: модель CMYK

CMYK — субтрактивная (вычитающая) модель, используемая в полиграфии. Компоненты Cyan Magenta Yellow описывают, какие красители поглощают соответствующие части спектра, а канал K (Key/Black) добавляют для глубокого чёрного, экономии краски и повышения контрастности печати. CMYK — модель устройства вывода: она зависит от конкретного принтера, бумаги и профиля, поэтому для точной цветопередачи применяется цветоуправление (ICC-профили).

Плюс CMYK — соответствие печатному процессу. Минус — меньший цветовой охват по сравнению с RGB и сильная зависимость от устройства.

27. Модели цветного изображения: модель HSV

HSV — перцептивно ориентированная модель, удобная для анализа и редактирования “человеческими” параметрами. Hue задаёт тон (какой цвет), Saturation — насыщенность (чистота цвета), Value — яркость/интенсивность. HSV часто применяют в компьютерном зрении для сегментации по цвету, потому что тон HHH и насыщенность SSS менее чувствительны к изменениям освещения, чем сами компоненты RGB (хотя полностью проблему освещения HSV не решает). Недостаток HSV — нелинейность и неоднородность: одинаковые изменения параметров HSV не всегда соответствуют одинаковым визуальным изменениям.

28. Модели цветного изображения: модель Lab

Lab — цветовая модель, построенная так, чтобы быть ближе к перцептивной равномерности, то есть чтобы одинаковое изменение координат примерно соответствовало одинаковому изменению воспринимаемого цвета. Компонента L* описывает светлоту, a* — ось “зелёный–красный”, b* — ось “синий–жёлтый”. Lab является аппаратно-независимой моделью и обычно используется как промежуточное пространство в системах цветоуправления и в задачах, где важны измеримые цветовые различия, например через ΔE.

Плюс Lab — отделение светлоты от цветовых компонент и близость к восприятию. Минус — вычислительная сложность преобразования и необходимость корректного белого/референса.

29. Модели цветного изображения: модель RGB

RGB — аддитивная (суммирующая) модель, в которой цвет задаётся интенсивностями трёх базовых компонент Red Green Blue. Она напрямую соответствует принципу работы экранов и большинства камер, поэтому является основной для хранения и отображения изображений. Пиксель представляют как тройку (R,G,B) обычно в диапазоне 0…255 (8 бит на канал) или нормированно 0…1. Сильная сторона RGB — простота и аппаратная “естественность”. Слабая сторона — то, что компоненты RGB сильно коррелированы и не отделяют яркость от цветности, поэтому многие задачи обработки (сегментация по цвету, коррекция освещённости, компрессия) в RGB делать неудобно.

30. Модели цветного изображения: модель YCbCr

YCbCr — модель, широко применяемая в видео и сжатии изображений. Компонента YYY — яркостная (luma), Cb и Cr— цветоразностные компоненты (сине-разностная и красно-разностная). Главное преимущество в том, что яркость отделена от цветности, и человеческое зрение менее чувствительно к деталям в цветности, чем в яркости, поэтому можно делать субдискретизацию цветовых каналов (например 4:2:0) почти без заметной потери качества — это основа JPEG/MPEG-подходов. Минус — это не полностью “физическая” яркость и конкретные формулы преобразования зависят от стандарта (например BT.601, BT.709).

31. Модели цветного изображения: трехкомпонентная теория цвета, оппонентная теория цвета

Суть трёхкомпонентной теории в том, что цветовое ощущение можно описать **тремя независимыми сигналами**, потому что в сетчатке есть **три типа колбочек** (условно чувствительные к коротким, средним и длинным длинам волн — S, M, L). Любой спектр света вызывает некоторую тройку откликов этих колбочек, и мозг интерпретирует её как определённый цвет. Отсюда следует практический вывод: для задания цвета в технике достаточно трёх чисел, поэтому работают модели типа **RGB** (три компоненты интенсивностей). Важное следствие —

Оппонентная теория говорит, что после первичного трихроматического кодирования зрительная система переходит к **каналам разностей**, где цвет представляется через противопоставления. Основные оппонентные пары — **красный–зелёный**, **синий–жёлтый**, а отдельно идёт канал **яркости** (светлота). Поэтому некоторые сочетания воспринимаются как “невозможные” (например,

Практически оппонентная модель лежит в основе пространств, где яркость отделена от цветности и цвет задаётся разностями, например **YCbCr** (яркость + два цветоразностных канала) и частично **Lab** (светлота L^* и две оппонентные оси a^*, b^*).

32. Байеровский паттерн. Способы восстановления цвета

Байеровский паттерн (Bayer CFA) — это цветовая мозаика на матрице камеры, где каждый фотосенсор измеряет только один канал R, G или B; чаще всего используется схема RGGB, при которой зелёных элементов в 2 раза больше, потому что через G лучше передаётся яркостная детальность. На выходе получается RAW-мозаика, поэтому для получения полного RGB-изображения выполняют демозаику (восстановление недостающих каналов). Самые простые способы — ближайший сосед и билинейная интерполяция; более качественные — методы, учитывающие границы (edge-directed/градиентные), а также адаптивные алгоритмы типа VNG и AHD и линейные с поправками (Malvar–He–Cutler). Цель демозаики — восстановить R, G, B в каждой точке с минимальными артефактами (ложный цвет, муар, “зубчики” на границах).

33. Баланс белого. Цветовой баланс

(до точки)

Баланс белого — это цветовая коррекция, цель которой сделать так, чтобы нейтральные объекты (белый и серый) на изображении оставались нейтральными при любом освещении, то есть имели равные значения каналов без цветного оттенка. Причина в том, что разные источники света имеют

Да, с помощью **серой/белой карточки** баланс белого делается самым корректным способом, потому что ты даёшь камере/программе **эталон нейтрали** при текущем освещении.

Как это работает по сути

Карточка должна быть **нейтральной** (без оттенка). В идеале это:

- **серая 18%** (gray card) — чаще для точной экспозиции и WB
- **белая карточка** — тоже подходит, но может клипповаться (пересветиться)

Дальше алгоритм простой: на карточке каналы должны стать равными, то есть $R = G = B$.

С научной точки зрения цветовой баланс включает несколько типов коррекций:

1. **Глобальные усиления каналов** (по сути продолжение WB):

$$R' = k_R R, \quad G' = k_G G, \quad B' = k_B B$$

Это сдвигает общий оттенок изображения и устраняет доминирующий цветовой налёт.

2. **Баланс по тоновым диапазонам** (тени–средние–света): корректировка может быть разной в тёмных, средних и светлых областях, чтобы, например, убрать синеву в тенях, не меняя светлые области. Это уже не одна пара коэффициентов, а разные преобразования для разных уровней яркости (часто кусочно-линейные или кривые).
3. **Цветовая трансформация матрицей** (более “физично”):

$$\begin{pmatrix} R' \\ G' \\ B' \end{pmatrix} = \mathbf{M} \begin{pmatrix} R \\ G \\ B \end{pmatrix}$$

34. Базовые алгоритмы обработки цветного изображения: модель “Серый мир”, линейное растяжение контраста

Это базовый алгоритм автоматического **баланса белого**.

- модель «Серого мира» (Grayworld)
  - средний уровень («серый») по каждому каналу должен быть одинаков для всех каналов
  - если цветовой баланс нарушен, тогда «серый» в этом канале больше «серого» других каналов
  - вычислим коэффициенты усиления так, чтобы среднее в каждом канале стало одинаковым:

$$\overline{R} = \frac{1}{N}\sum R(x,y); \qquad \overline{G} = \frac{1}{N}\sum G(x,y); \qquad \overline{B} = \frac{1}{N}\sum B(x,y); \qquad Avg = \frac{\overline{R}+\overline{G}+\overline{B}}{3}$$

$$R' = R \cdot \frac{Avg}{\overline{R}}; \qquad G' = G \cdot \frac{Avg}{\overline{G}}; \qquad B' = B \cdot \frac{Avg}{\overline{B}}$$

Линейное растяжение контраста

- Суть этого метода – линейно растянуть интенсивности по каждому цветовому каналу на весь динамический диапазон.
- Алгоритм линейного растяжения контраста
  - Найти минимум и максимум изображения по каждому из каналов (R_{min}, R_{max}, G_{min}, G_{max}, B_{min}, B_{max})
  - Преобразовать оттенки цвета каждого пиксела:

$$R = (R - R_{min}) \cdot \frac{255-0}{R_{max} - R_{min}} \qquad G = (G - G_{min}) \cdot \frac{255-0}{G_{max} - G_{min}} \qquad B = (B - B_{min}) \cdot \frac{255-0}{B_{max} - B_{min}}$$

35. Коррекция цвета на цветных изображениях

Коррекция цвета на цветных изображениях — это приведение цветопередачи к правильной или требуемой, то есть устранение цветового сдвига и настройка яркости/насыщенности. Базовый шаг — **баланс белого**, обычно как масштабирование каналов $R' = k_R R,\ G' = k_G G,\ B' = k_B B$, чтобы нейтральные области стали нейтральными $R' \approx G' \approx B'$ (коэффициенты берут по серой/белой области или автоматически, например “серый мир”). Дальше часто выполняют **тональную коррекцию** по яркостной компоненте (в пространствах YCbCr или Lab корректируют Y или L^*, чтобы не менять оттенки). Для более точной калибровки применяют **матричную цветокоррекцию** $[R', G', B']^T = \mathbf{M}[R, G, B]^T$, которая учитывает перекрёстное влияние каналов. Для финальной настройки используют **нелинейные преобразования** (гамма, кривые, LUT) и регулировку **насыщенности/оттенка** (обычно в HSV/HSL или Lab).

36. Типы шумов на цифровом изображении

- коррелированный или некоррелированный
- зависящий или не зависящий от самого регистрируемого сигнала

Импульсный шум:

- это искажение сигнала импульсами, т.е. выбросами с очень большими положительными или отрицательными значениями и малой длительностью
- часто возникает вследствие ошибок декодирования
- заметны на очень темных или очень светлых участках изображений

Аддитивный шум описывается соотношением

$$g(x, y) = f(x, y) + \eta(x, y)$$

где

- $f(x, y)$ - исходное изображение;
- $g(x, y)$ - зашумленное изображение;
- $\eta(x, y)$ - аддитивный и не зависящий от сигнала шум с гауссовым или другим распределением функции плотности вероятности.

Гауссов шум (называемый также **нормальным шумом**) возникает на изображении в результате проявления таких факторов, как шум в электрических цепях, шум сенсоров (из-за недостатка освещения и/или высокой температуры). Модель этого

Мультипликативный шум описывается соотношением

$$g(x, y) = f(x, y) \cdot \eta(x, y)$$

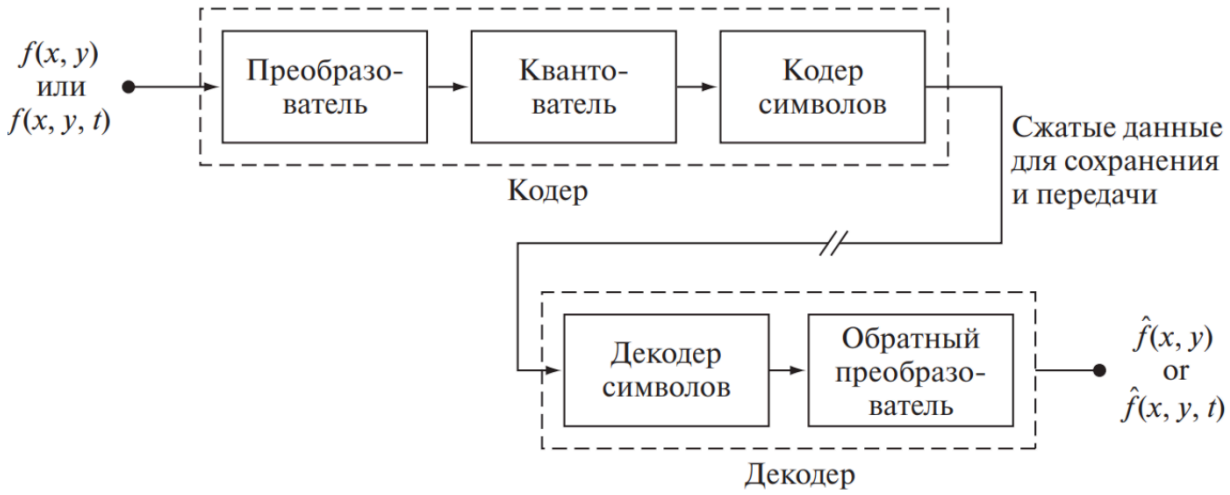
т. е. шумовая составляющая умножает принятый сигнал. Примерами такого шума являются: зернистость фотопленки, изображения, полученные с помощью ультразвука; радиолокационные изображения.

Квантовый шум (ошибка квантования) — возникает при переводе аналогового сигнала в дискретные уровни яркости и при ограниченной битовой глубине. Проявляется как ступенчатость градиентов и мелкие искажения, особенно на плавных переходах (постеризация).

37. Методы сжатия: общая информация, модель сжатия/восстановления

Методы сжатия данных предназначены для уменьшения объёма хранения и передачи информации за счёт устранения избыточности.

- Методы сжатия изображений можно разбить на две группы (+1):
  - сжатие без потерь, позволяющее точно восстанавливать исходное изображение;
  - сжатие с потерями (или сжатие с регулируемой степенью искажений), изображение восстанавливается приближенно;
  - сжатие почти без потерь



38. Методы сжатия без потерь: алгоритм Хаффмана

Алгоритм Хаффмана — классический метод **сжатия без потерь**, относящийся к энтропийному кодированию. Его идея в том, чтобы присвоить более частым символам более короткие двоичные коды, а редким — более длинные, при этом коды строятся так, чтобы их можно было однозначно декодировать (префиксные коды).

Сначала по данным строят алфавит символов и оценивают их частоты p_i. Затем создают дерево Хаффмана: на каждом шаге выбирают два символа или узла с минимальными частотами, объединяют их в новый узел с весом, равным сумме частот, и повторяют процесс до тех пор, пока не останется один корневой узел. После построения дерева всем ветвям приписывают биты, обычно “0” и “1”, и код каждого символа получается как последовательность битов вдоль пути от корня к листу. Полученный код является префиксным, то есть ни один код не является началом другого, поэтому декодирование выполняется однозначно при последовательном чтении битов.

Средняя длина кода равна $\bar{L} = \sum p_i \, l_i$, где l_i — длина кода символа, и для кода Хаффмана она близка к энтропии источника $H = -\sum p_i \log_2 p_i$ и удовлетворяет оценке $H \le \bar{L} < H + 1$.

- Применяется в
  - CCITT
  - JBIG2
  - JPEG
  - MPEG-1, 2, 4

39. Методы сжатия без потерь: коды Голомба

Коды Голомба — это префиксное кодирование **неотрицательного целого** n при заданном параметре $m > 0$. Код $G_m(n)$ строится как **унарный код частного** и **усечённый двоичный код остатка**.

Сначала вычисляют

$$q = \left\lfloor \frac{n}{m} \right\rfloor, \qquad r = n \bmod m.$$

Унарная часть для q: это q единиц и завершающий ноль (то есть 1^q0).

Далее задают параметры усечённого двоичного кода:

$$k = \lceil \log_2 m \rceil, \qquad c = 2^k - m.$$

Остаток кодируют по правилу:

$$\text{если } r < c, \text{ то } r \text{ кодируют в } (k-1) \text{ битах;}$$

$$\text{если } r \geq c, \text{ то } r' = r + c \text{ и } r' \text{ кодируют в } k \text{ битах.}$$

Эквивалентно можно записать

$$r' = \begin{cases} r, & r < c \\ r + c, & r \geq c \end{cases}$$

(при этом длина двоичной части равна $k-1$ или k).

Итоговый код Голомба:

$$G_m(n) = \mathrm{Unary}(q) \parallel \mathrm{TruncBin}(r),$$

то есть конкатенация унарного кода частного и усечённого двоичного кода остатка.

- Применяется в
  - JPEG-LS
  - AVS

40. Методы сжатия без потерь: арифметическое кодирование

Арифметическое кодирование — это сжатие без потерь, где кодируют не каждый символ отдельным кодом, а **всю последовательность одним числом** (интервалом) на $[0, 1)$. По вероятностной модели каждому символу дают подинтервал, и при чтении символов текущий интервал $[L, U)$ последовательно **сужают**; после конца сообщения передают любое число внутри итогового интервала. Декодер, зная число и модель вероятностей, по попаданию в подинтервалы восстанавливает символы. Суть: чем вероятнее последовательность, тем уже итоговый интервал и тем меньше бит нужно; метод обычно эффективнее Хаффмана и ближе к энтропийной границе.

- используется в
  - JBIG1
  - JBIG2
  - JPEG-2000
  - H.264
  - MPEG-4 AVC
- и других стандартах сжатия

41. Методы сжатия без потерь: алгоритмы RLE и LZW



Алгоритм RLE

3 Сжатие без потерь

- Изображение вытягивается в цепочку байт по строкам растра.
- Само сжатие в RLE происходит за счет того, что в исходном изображении встречаются цепочки одинаковых байт. Замена их на пары <счетчик повторений, значение> уменьшает избыточность данных.

- используется в
  - CCITT
  - JBIG2
  - JPEG
  - M-JPEG
  - MPEG-1,2,4
  - BMP

- Алгоритм LZW (от фамилий авторов Lempel - Ziv-Welch) является более сложным и универсальным алгоритмом (по отношению к RLE), сжимающим изображения за счет поиска, выделения и кодирования одинаковых подцепочек в потоке байтов.
- used in the
  - GIF
  - TIFF
  - PDF

Мы читаем вход последовательно и держим текущую строку w. Берём следующий символ k и проверяем, есть ли строка $w + k$ в словаре. Если $w + k$ есть, то просто расширяем $w := w + k$ и читаем дальше. Если $w + k$ нет, то выводим код для текущей строки w, добавляем в словарь новую строку $w + k$ и начинаем заново с $w := k$. В конце потока выводим код последней строки w.

42. Методы сжатия с потерями: JPEG

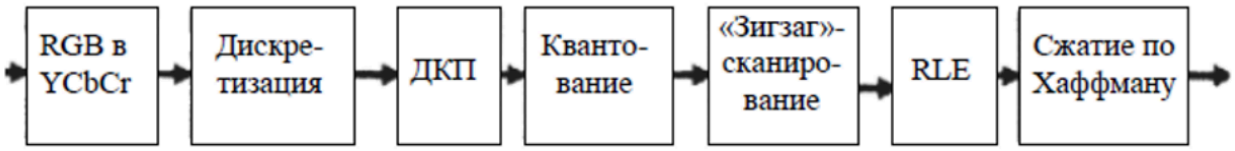


Схема преобразования данных в алгоритме сжатия 24-битового цветного изображения JPEG

JPEG — это классический метод **сжатия с потерями** для фотографий, основанный на том, что изображение переводят в частотную область, где высокочастотные компоненты можно сильнее “урезать” без сильного ухудшения восприятия.

Схема JPEG обычно такая. Сначала изображение переводят из RGB в пространство **YCbCr**, чтобы отделить яркость Y от цветности Cb, Cr. Затем цветовые каналы часто **субдискретизируют** (например 4:2:0), потому что глаз менее чувствителен к деталям в цветности. После этого изображение разбивают на блоки **8×8**, и для каждого блока вычисляют **2D DCT** (дискретное косинусное преобразование), получая коэффициенты частот. Далее выполняют ключевой шаг — **квантование** коэффициентов DCT по таблице: делят коэффициенты на значения квантования и округляют. Именно квантование даёт потери и обеспечивает основное сжатие, причём высокие частоты квантируются сильнее. После квантования коэффициенты упорядочивают зигзагом, чтобы сгруппировать нули, применяют **RLE** для серий нулей и затем энтропийное кодирование (обычно Хаффман). На декодере всё делается в обратном порядке: декодирование, обратное квантование, обратное DCT, сборка блоков и перевод обратно в RGB.

43. Методы сжатия с потерями: JPEG2000

JPEG2000 — стандарт **сжатия с потерями (и без потерь тоже)**, который в отличие от JPEG использует не блочное DCT, а **вейвлет-преобразование (DWT)** и поэтому обычно даёт лучшее качество на высоких степенях сжатия и меньше “квадратиков”.

Схема JPEG2000 в общих чертах такая. Сначала изображение (часто после преобразования цветового пространства, например в YCbCr/аналог) разбивают на **тайлы** (не обязательно, но часто), после чего к каждому тайлу применяют **2D дискретное вейвлет-преобразование (DWT)**, получая набор поддиапазонов частот на нескольких масштабах (LL, LH, HL, HH и дальше рекурсивно). Для сжатия с потерями коэффициенты DWT **квантуются** (обычно скалярное квантование с шагом), причём потери управляются шагом квантования и целевым битрейтом. Затем коэффициенты кодируются очень эффективно: изображение поддиапазонов разбивают на **code-blocks**, и применяется алгоритм **EBCOT** (Embedded Block Coding with Optimized Truncation) — по сути побитово-плоскостное кодирование (bit-plane coding) с **контекстным арифметическим кодированием** и возможностью оптимально “обрезать” поток под заданный размер/качество. За

44. Методы сжатия видеопоследовательностей: общие сведения, кодека

Сжатие видеопоследовательностей основано на устранении пространственной избыточности внутри кадра и временной избыточности между кадрами. В современных кодеках применяется гибридная схема: блоковое предсказание (intra — по соседним пикселям текущего кадра, inter — по опорным кадрам с оценкой и компенсацией движения) и кодирование ошибки предсказания. Остаток (residual) подвергается преобразованию (DCT-подобному/целочисленному), затем квантованию, которое задаёт потери и степень сжатия, после чего выполняется энтропийное кодирование. Поток организуется через типы кадров: I-кадры кодируются независимо, P-кадры — по предсказанию от прошлого, B-кадры — по двунаправленному предсказанию от прошлого и будущего, что повышает эффективность при увеличении сложности и задержки. Кодек — это совокупность инструментов и правил формирования/декодирования битового потока (например H.264/AVC, HEVC, AV1), обеспечивающая компромисс между битрейтом, качеством, вычислительной сложностью и задержкой; контейнер (MP4, MKV) лишь упаковывает эти потоки.

45. Нанесение водяных цифровых знаков

Нанесение цифрового водяного знака — это встраивание в изображение/видео скрытой или видимой метки для защиты авторства, отслеживания и проверки целостности. Общая модель: исходный сигнал x, знак w, ключ k; встраивание $y = E(x, w, k)$, извлечение или проверка $\hat{w} = D(y, k)$ / $V(y, k)$. Метки бывают видимые и невидимые, а также робастные (переживают JPEG/фильтрацию) и хрупкие (разрушаются при изменениях). Встраивание делают либо в пространственной области (например LSB — запись в младшие биты пикселей), либо чаще в частотной (DCT/DWT — изменение коэффициентов, обычно средних частот) для лучшей незаметности и устойчивости. Детектирование бывает blind (без оригинала) и non-blind (с оригиналом); часто используют корреляционную проверку по шаблону, заданному ключом.

46. Математическая морфология бинарных изображений: эрозия и дилатация, размыкание и замыкание, операция утоньшения

Математическая морфология бинарных изображений рассматривает изображение как множество пикселей объекта $A \subset \mathbb{Z}^2$ и использует структурный элемент B (малое множество точек), который “сканирует” изображение.

Дилатация (расширение) увеличивает объект, добавляя пиксели там, где структурный элемент пересекается с объектом:

$$A \oplus B = \{ z \mid (\hat{B})_z \cap A \neq \varnothing \},$$

где $\hat{B}$ — отражение B, а $(\hat{B})_z$ — сдвиг на z. Интуитивно дилатация “наращивает” границы и заполняет мелкие разрывы.

Эрозия (сжатие) уменьшает объект, оставляя только те точки, где структурный элемент целиком помещается внутри объекта:

$$A \ominus B = \{ z \mid B_z \subseteq A \}.$$

Интуитивно эрозия “съедает” тонкие выступы и убирает мелкие объекты.

Размыкание (открытие) — это эрозия, затем дилатация тем же структурным элементом:

$$A \circ B = (A \ominus B) \oplus B.$$

Открытие удаляет мелкие объекты и выступы, сглаживает контур без сильного увеличения объекта.

Замыкание (закрытие) — это дилатация, затем эрозия:

$$A \bullet B = (A \oplus B) \ominus B.$$

Закрытие заполняет мелкие отверстия и разрывы, соединяет близкие компоненты и сглаживает контур.

Утоньшение (thinning) — операция, уменьшающая толщину объекта при сохранении его связности и топологии; применяется для получения “скелета” или тонких линий. Формально утоньшение часто задают через последовательное удаление граничных пикселей, которые удовлетворяют маскам, либо через морфологическое представление:

$$A \otimes B = A \setminus (A \circledast B),$$

47. Математическая морфология бинарных изображений: преобразование "Hit-or-Miss"

Преобразование **Hit-or-Miss (попадание–промах)** — базовая операция математической морфологии для бинарных изображений, предназначенная для **поиска (детектирования) заданного шаблона** формы в изображении. В отличие от дилатации/эрозии, которые “расш

Пусть $A \subset \mathbb{Z}^2$ — множество пикселей объекта (1), $\overline{A}$ — фон (0). Шаблон задаётся парой структурных элементов B_1 и B_2, где B_1 описывает, где должны быть единицы (объект), а B_2 — где должны быть нули (фон). Тогда преобразование hit-or-miss определяется как

$$A \circledast (B_1, B_2) = (A \ominus B_1) \cap (\overline{A} \ominus B_2).$$

Смысл формулы такой: точка попадает в результат тогда, когда B_1 полностью помещается в объект A (это “hit”), и одновременно B_2 полностью помещается во фон $\overline{A}$ (это “miss”). То есть локально вокруг точки изображение совпадает с заданным двоичным шаблоном.

---

48. Сегментация изображения: общие сведения, модели перепадов

Сегментация — это разбиение изображения на области или выделение объектов, то есть построение карты меток $L(x, y)$, где каждый пиксель отнесён к объекту/фону или к одному из классов. В основе сегментации два принципа: внутри области пиксели должны быть однородны (по яркости, цвету, текстуре), а между областями есть различие, которое проявляется как граница. Поэтому методы обычно опираются либо на однородность (пороговая сегментация, рост областей, кластеризация), либо на границы (поиск контуров по производным).

Модели перепадов описывают, как выглядит граница по яркости. Основная модель — ступенчатый перепад (резкий переход между двумя уровнями), но в реальности он сглаживается и становится “размытой ступенькой” из-за оптики и шума. Есть также модель рампы, когда переход идёт на некоторой ширине, и модель “крыши” (roof), характерная для тонкой линии или полосы, где яркость растёт до пика и падает. Из этих моделей следует, что границы удобно находить по максимумам градиента (первая производная) или по нулевым пересечениям второй производной (Лаплас), обычно после предварительного сглаживания.

- Перепад - связное множество пикселей, лежащих на границе между двумя областями.

---

49. Обнаружение точек, линий и перепадов. Пороговая обработка

Обнаружение точек, линий и перепадов в изображении обычно строится на локальных операторах, которые усиливают нужный тип структуры, после чего применяется пороговая обработка для выделения найденных элементов. Точку понимают как локальную аномалию яркости относительно окружения, линию — как вытянутую структуру с повышенной или пониженной яркостью, а перепад (границу) — как резкое изменение яркости между областями. Для их обнаружения используют свёртку с масками производных или масками согласования: для точек — маски, подобные дискретному лапласиану, которые дают большой отклик на изолированные изменения; для линий — направленные линейные маски (горизонтальные, вертикальные, диагональные), максимальный отклик которых указывает ориентацию линии; для перепадов — градиентные операторы (Робертс, Превитт, Собель) по первой производной, либо Лаплас/LoG по второй производной с нулевыми пересечениями.

Пороговая обработка — это этап принятия решения, где по значению отклика оператора формируется бинарный результат. Обычно задают порог T и считают, что структура обнаружена, если модуль отклика превышает порог: $g(x, y) = 1$, если $|R(x, y)| \geq T$, иначе $g(x, y) = 0$, где $R(x, y)$ — отклик фильтра. Для перепадов часто порогуют модуль градиента, для точек — величину отклика точечного детектора, для линий — максимум откликов по ориентациям. Порог может быть глобальным (один на всё изображение) или адаптивным (зависит от локальной статистики), что важно при неравномерном освещении и шуме.

---

50. Сегментация на отдельные области: выращивание областей

- **Выращивание областей** представляет собой процедуру, которая группирует пиксели или подобласти в более крупные области по заранее заданным критериям.

Выращивание областей (region growing) — это алгоритм сегментации, который начинается с одного или нескольких “семян” (seed-пикселей) и расширяет каждую область, последовательно добавляя к ней соседние пиксели. Задают тип соседства (4 или 8) и критерий однородности, например чтобы яркость/цвет кандидата отличались от текущего среднего области не больше чем на порог T. Далее семя помещают в область и в очередь, затем циклически берут пиксель из очереди, проверяют его соседей и каждого ещё не размеченного соседа, удовлетворяющего критерию, присоединяют к области и добавляют в очередь, одновременно обновляя параметры области (среднее, дисперсию и т.п.). Процесс продолжается, пока очередь не опустеет, то есть пока область перестанет расти; затем аналогично обрабатывают остальные семена, получая набор связных областей.

Критерий однородности в алгоритме выращивания областей — это формальное условие, по которому решается, можно ли присоединить соседний пиксель p к текущей области R. Смысл критерия в том, что присоединяемый пиксель должен быть “достаточно похож” на уже сформированную область по выбранному признаку (яркость, цвет, текстура), а степень похожести задаётся порогом. Наиболее распространённый вариант для полутонового изображения — сравнение с текущим средним области: пиксель добавляют, если $|f(p) - \mu_R| \le T$, где $f(p)$ — яркость пикселя, μ_R — средняя яркость области, T — порог. Более локальный вариант использует разность с граничным пикселем области q: $|f(p) - f(q)| \le T$, что проще, но может “протекать”

---

51. Сегментация на отдельные области: разделение и слияние областей

Разделение и слияние областей (split-and-merge) — это метод сегментации, который строит области, опираясь на критерий однородности, но делает это не “от семян” как выращивание, а через управляемое разбиение и объединение. Идея такая: если область неоднородна, её нужно разделить; если две соседние области вместе однородны, их нужно слить.

Алгоритм обычно описывают так. Сначала всё изображение рассматривают как одну область R. Проверяют критерий однородности $P(R)$ (например, дисперсия яркости внутри области меньше порога). Если $P(R)$ не выполняется, область делят на 4 подобласти (квадранты) — это разбиение по схеме квадродерева (quadtree). Затем для каждой из полученных областей снова проверяют P и продолжают разбиение рекурсивно, пока для всех областей критерий однородности не станет выполненным или пока не будет достигнут минимальный размер блока.

После стадии разделения выполняют стадию слияния. Рассматривают пары соседних областей R_i и R_j. Если объединённая область $R_i \cup R_j$ удовлетворяет критерию однородности $P(R_i \cup R_j)$, то эти области объединяют. Слияние продолжают итеративно, пока больше нельзя объединить ни одну пару соседних областей без нарушения критерия.

Суть метода: разделение гарантирует, что внутри областей станет “достаточно однородно”, а слияние устраняет излишнее дробление и восстанавливает более крупные связные области. Результат зависит от выбора критерия однородности и порогов, а также от правила соседства (обычно 4- или 8-соседство).

---

52. Сегментация областей с использованием графовых разрезов

Сегментация с использованием графовых разрезов (graph cuts) — это метод, в котором изображение представляют как граф и находят разбиение на области как задачу минимизации энергии. Пиксели (или суперпиксели) являются вершинами графа, а рёбра задают связи между соседними пикселями. Обычно добавляют два специальных узла: **источник** s (объект, foreground) и **сток** t (фон, background). Тогда сегментация сводится к поиску **минимального s–t разреза**, который разделяет вершины на два множества S и T, соответствующие объекту и фону.

Целевая функция обычно имеет вид:

$$E(L) = \sum_{p} D_p(L_p) + \sum_{(p,q)\in\mathcal{N}} V_{pq}(L_p, L_q),$$

где $L_p \in \{\text{obj}, \text{bg}\}$ — метка пикселя p. Член D_p (data term) задаёт, насколько пиксель похож на модель объекта или фона (например по цвету/яркости), а член V_{pq} (smoothness term) штрафует разрывы между соседями и делает границы более гладкими, обычно завися от контраста между пикселями (через него границы “охотнее” проходят по сильным перепадам).

Как строят граф. Каждому пикселю p задают ребро к источнику с весом, связанным с $D_p(\text{bg})$, и ребро к стоку с весом, связанным с $D_p(\text{obj})$ (конкретное соответствие зависит от принятой записи, но смысл один: дороже “отрезать” пиксель от той метки, которой он хорошо соответствует). Между соседними пикселями p, q ставят ребро с весом w_{pq}, который обычно тем больше, чем больше мы хотим удерживать их в одной области; часто берут

$$w_{pq} = \lambda \exp\left(-\frac{\|I_p - I_q\|^2}{2\sigma^2}\right),$$

то есть если пиксели похожи, ребро сильное и разрез “не любит” проходить между ними, а если сильно различаются, ребро слабое и разрез легче проходит по границе.

Дальше находят минимальный разрез, что эквивалентно задаче максимального потока (max-flow/min-cut) и решается полиномиально. Результат — разметка: вершины, оказавшиеся на стороне источника, относятся к объекту, на стороне стока — к фону. Метод особенно эффективен, когда

---

53. Сегментация по морфологическим водоразделам

Сегментация по морфологическим водоразделам (watershed) — это метод, который рассматривает изображение как топографическую поверхность: яркость (или градиент) трактуется как “высота”. Тогда локальные минимумы — это “впадины”, а границы объектов соответствуют “водоразделам” между бассейнами.

На практике чаще всего строят водораздел **по модулю градиента** $G(x, y)$, потому что границы объектов дают большие значения градиента, и водораздел естественно проходит по этим “хребтам”. Алгоритмически можно объяснить так: если “заливать” поверхность водой, начиная из всех минимумов одновременно, то области, которые заполняются из одного минимума, образуют сегменты (водосборные бассейны), а там, где фронты воды встречаются, формируются линии водораздела — границы сегментов.

Классическая проблема watershed — **пересегментация**: из-за шума и мелких неоднородностей появляется слишком много локальных минимумов, и изображение разбивается на множество мелких областей. Поэтому в морфологической практике почти всегда используют **маркерный водораздел**. Сначала формируют маркеры объектов и фона (например, через морфологическое

54. Признаки, используемые для описания объектов. Представление объектов в виде вектора признаков

Наиболее распространены геометрические (морфологические) признаки: площадь S как число пикселей области, периметр P как длина контура, центр масс $(\bar{x}, \bar{y})$, ограничивающий прямоугольник, главные оси и ориентация, эксцентриситет, компактность, например $C = \frac{P^2}{4\pi S}$, а также моменты (обычные, центральные, нормированные), включая инварианты к сдвигу, масштабу и повороту. Для объектов с выраженной границей применяют контурные признаки: цепной код, кривизна, углы, сигнатуры расстояния до центра, а также дескрипторы формы типа Фурье-дескрипторов. Для объектов с внутренней структурой используют текстурные признаки: статистики (среднее, дисперсия), матрицы сопряжённости (GLCM), локальные бинарные шаблоны (LBP), отклики фильтров (Габор), а также частотные признаки. Для цветных объектов добавляют цветовые признаки: средние значения каналов в выбранном пространстве (RGB/HSV/Lab), гистограммы цвета, моменты распределения цвета. В задачах с движением или видео используют динамические признаки, например векторы движения и траектории.

Представление объекта в виде вектора признаков означает, что для каждого объекта формируется упорядоченный набор чисел:

$$\mathbf{f} = [f_1, f_2, \ldots, f_d]^T,$$

где каждое f_i — выбранный признак (например площадь, периметр, средняя яркость, параметр текстуры и т.д.), а d — размерность пространства признаков. Такой вектор используется далее в алгоритмах классификации и распознавания: объекты сравнивают по расстоянию между векторами, либо подают в обучаемую модель (k-NN, SVM, нейросеть). На практике важны

55. Дескрипторы признаков границ. BRIEF

В задачах анализа изображений “признак границы” обычно означает **ключевую точку на контуре** (угол, конец линии, резкий изгиб) или локальный фрагмент границы. Чтобы потом сравнивать такие точки между изображениями, для каждой точки строят **дескриптор** — компактное численное описание локального окружения, которое должно быть устойчивым к шуму и небольшим изменениям масштаба/поворота/освещения.

Классические дескрипторы границ и контуров делятся на две группы:

- **Контурные (shape) дескрипторы**, которые описывают форму самой границы: цепной код, кривизна, сигнатура расстояний до центра, Fourier-дескрипторы, shape context и т.п.
- **Дескрипторы локального патча вокруг точки границы**, которые описывают не только линию, но и яркостную структуру рядом с ней (градиенты/текстуры). К ним относятся SIFT/SURF/HOG-подобные и **бинарные дескрипторы** (BRIEF, ORB, BRISK, FREAK).

BRIEF — это **бинарный дескриптор** локального патча вокруг ключевой точки. Он кодирует структуру изображения не градиентами, а набором очень простых тестов: *сравнение яркости двух точек* в окне.

Идея BRIEF

Берём патч (обычно вокруг ключевой точки), предварительно слегка **сглаживаем** (чтобы уменьшить шум), и выполняем N парных сравнений интенсивности:

$$\tau(p_i, q_i) = \begin{cases} 1, & I(p_i) < I(q_i) \\ 0, & \text{иначе} \end{cases}$$

Каждое сравнение даёт 1 бит. В итоге дескриптор — это бинарная строка длины N бит (часто 128, 256 или 512).

---

56. Метод главных компонент

Метод главных компонент (PCA) — это линейный метод уменьшения размерности, который заменяет исходные признаки набором новых взаимно ортогональных признаков (главных компонент), упорядоченных по убыванию дисперсии. Идея: найти такие направления в пространстве данных, вдоль которых разброс (информация) максимален, и проецировать данные на первые k направлений, сохраняя максимум вариативности при меньшей размерности.

Пусть есть матрица данных $X \in \mathbb{R}^{n \times d}$, где n объектов и d признаков. Сначала признаки центрируют: $X_c = X - \mathbf{1}\mu^T$, где μ — вектор средних. Далее строят ковариационную матрицу

$$C = \frac{1}{n-1} X_c^T X_c.$$

Затем находят собственные значения и собственные векторы: $Cv_i = \lambda_i v_i$. Собственные векторы v_i — это главные направления (компоненты), а λ_i — дисперсия данных вдоль них. Отсортировав λ_i по убыванию, берут первые k векторов и формируют матрицу $W_k = [v_1, \ldots, v_k]$. Проекция данных в новое пространство:

$$Z = X_c W_k,$$

где $Z \in \mathbb{R}^{n \times k}$ — представление данных в k-мерном пространстве главных компонент. При необходимости приближённое восстановление:

$$\hat{X} = Z W_k^T + \mathbf{1}\mu^T.$$

Основные применения PCA: сжатие признаков, устранение корреляций (декорреляция), подавление шума, визуализация многомерных данных, подготовка признаков для классификации. В обработке изображений PCA известен как “eigenfaces” для распознавания лиц и как способ построения компактных признаков.

---

57. Масштабно-инвариантное преобразование признаков (алгоритм SIFT)

SIFT (Scale-Invariant Feature Transform) — алгоритм поиска и описания ключевых точек, устойчивый к изменению масштаба и поворота и относительно устойчивый к изменению освещения. Он состоит из двух частей: детектор (где найти точки) и дескриптор (как их описать).

Сначала строят **масштабное пространство**: изображение сглаживают гауссовыми фильтрами с разными σ, получая $L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$. Затем вычисляют разности гауссиан (DoG) $D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$, которые приближают лапласиан гауссианы. Ключевые точки находят как локальные экстремумы DoG в 3D (по x, y и масштабу): пиксель сравнивают с соседями в текущем масштабе и в соседних масштабах. После этого выполняют уточнение положения точки (обычно через квадратичную аппроксимацию), отбрасывают точки с малым контрастом и точки, лежащие на ребре (по критерию отношения собственных значений матрицы Гессе).

Далее каждой точке назначают **ориентацию**, чтобы получить поворотную инвариантность. В окрестности точки на соответствующем масштабе вычисляют градиенты, строят гистограмму направлений, и доминирующее направление становится ориентацией ключевой точки (иногда допускают несколько ориентаций для одной точки).

После этого строят **дескриптор**. Вокруг точки берут окно (пропорциональное масштабу), делят на сетку 4×4 ячеек и для каждой ячейки строят гистограмму направлений градиента (обычно 8 бинов). Все гистограммы объединяют в вектор длины $4 \cdot 4 \cdot 8 = 128$. Затем вектор нормируют (и часто “обрезают” большие значения и снова нормируют), что повышает устойчивость к изменению освещения и контраста.

---

58. Устойчивые ускоренные признаки (алгоритм SURF)

SURF (Speeded-Up Robust Features) — алгоритм обнаружения и описания ключевых точек, задуманный как более быстрый аналог SIFT при сохранении устойчивости к масштабу, повороту и умеренным изменениям освещения. Ускорение достигается за счёт использования интегральных изображений и приближённых фильтров.

В детекторе SURF строит масштабное пространство не через разности гауссиан, а через приближение **детерминанта Гессе** (Hessian) на разных масштабах. Для точки (x, y) матрица Гессе H содержит вторые производные, а “сила” интересной точки оценивается детерминантом $\det(H)$. Вместо точных производных SURF использует **box-фильтры** (прямоугольные маски), которые быстро считаются через **интегральное изображение**, поэтому вычисления значительно ускоряются. Экстремумы $\det(H)$ ищутся в пространстве (x, y, scale), что даёт масштабную инвариантность.

Для поворотной инвариантности SURF назначает ключевой точке ориентацию: в окрестности вычисляют отклики Хаара (Haar wavelets) по x и y, и доминирующее направление этих откликов выбирают как ориентацию. Далее строится дескриптор: окрестность ориентируют по найденному углу, делят на подблоки (обычно 4×4), в каждом суммируют отклики Хаара $\sum d_x$, $\sum d_y$, $\sum |d_x|$, $\sum |d_y|$. Эти значения объединяются в вектор признаков. Классический дескриптор SURF имеет размерность 64 (есть расширенная версия 128).

---

59. Детекторы углов Харриса и Моравеца

По-другому можно сказать, что углы - это точка, у которой в окрестности интенсивность изменяется относительно центра (x, y).
Углы определяются по координатам и изменениям яркости окрестных точек изображения.

- главное свойство таких точек заключается в том, что в области вокруг угла у градиента изображения преобладают два доминирующих направления, что делает их различимыми;

Детектор Моравеца

Идея: угол — это точка, где при небольшом сдвиге окна **в любом направлении** яркость сильно меняется. Для пикселя (x, y) берут окно W и считают "ошибку сдвига" для нескольких направлений (u, v):

$$E_{u,v}(x, y) = \sum_{(i,j)\in W} \big(I(i, j) - I(i + u, j + v)\big)^2.$$

Далее берут минимальное по направлениям:

$$R(x, y) = \min_{(u,v)\in\mathcal{D}} E_{u,v}(x, y),$$

где $\mathcal{D}$ — набор дискретных сдвигов (обычно 8 направлений). Если $R(x, y)$ большое — точка считается углом. Затем применяют порог и подавление немаксимумов.

Смысл: на плоской области R маленькое, на границе маленькое (если сдвиг вдоль границы), а на угле большое во всех направлениях.

Детектор Харриса (Harris–Stephens)

Харрис — развитие идеи Моравеца, но с **непрерывной** аппроксимацией малых сдвигов через градиенты. Рассматривают ошибку сдвига:

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \mathbf{M} \begin{bmatrix} u \\ v \end{bmatrix},$$

где $\mathbf{M}$ — матрица вторых моментов (structure tensor):

$$\mathbf{M} = \sum_{(i,j)\in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \quad (\text{часто с гауссовым весом}).$$

Собственные значения λ_1, λ_2 матрицы $\mathbf{M}$ интерпретируются так:

- λ_1, λ_2 маленькие → плоская область,
- одно большое, другое маленькое → граница,
- оба большие → угол.

---

60. Искусственные нейронные сети: нейронная сеть Хопфилда

Сеть Хопфилда — это рекуррентная нейросеть, которая работает как **ассоциативная память**: если подать на вход “шумный” или неполный образ, сеть итеративно приходит к ближайшему сохранённому шаблону. Она состоит из N нейронов со состояниями $s_i \in \{-1, +1\}$, веса симметричны $w_{ij} = w_{ji}$, самосвязи обычно отсутствуют $w_{ii} = 0$. Запоминание P образов $\xi^{(p)}$ часто делают по правилу Хебба:

$$w_{ij} = \frac{1}{N} \sum_{p=1}^{P} \xi_i^{(p)} \xi_j^{(p)}, \quad i \neq j, \qquad w_{ii} = 0.$$

Динамика задаётся обновлением нейронов (чаще асинхронно):

$$s_i \leftarrow \operatorname{sign}\left(\sum_j w_{ij} s_j - \theta_i \right),$$

где θ_i — порог (часто 0). Важное свойство сети — наличие **энергетической функции**

$$E(\mathbf{s}) = -\frac{1}{2} \sum_{i \neq j} w_{ij} s_i s_j + \sum_i \theta_i s_i,$$

которая при асинхронных обновлениях не возрастает, поэтому сеть гарантированно сходится к устойчивому состоянию (локальному минимуму энергии), интерпретируемому как восстановленный образ. Ограничения: ёмкость памяти порядка $P \approx 0.1N$–$0.14N$ для случайных образов, возможны ложные аттракторы и зависимость результата от начального состояния.

61. Искусственные нейронные сети: персептрон, многослойная сеть прямого распространения

Персептрон — простейшая нейросетевая модель для **бинарной классификации**. Он вычисляет взвешенную сумму признаков и пропускает её через пороговую функцию:

$$y = \text{sign}(\mathbf{w}^T\mathbf{x} + b),$$

где $\mathbf{x}$ — вектор признаков, $\mathbf{w}$ — веса, b — смещение. Геометрически это линейный классификатор: он разделяет классы гиперплоскостью. Обучение выполняют правилом коррекции ошибки (perceptron learning rule): если объект классифицирован неверно, веса корректируют

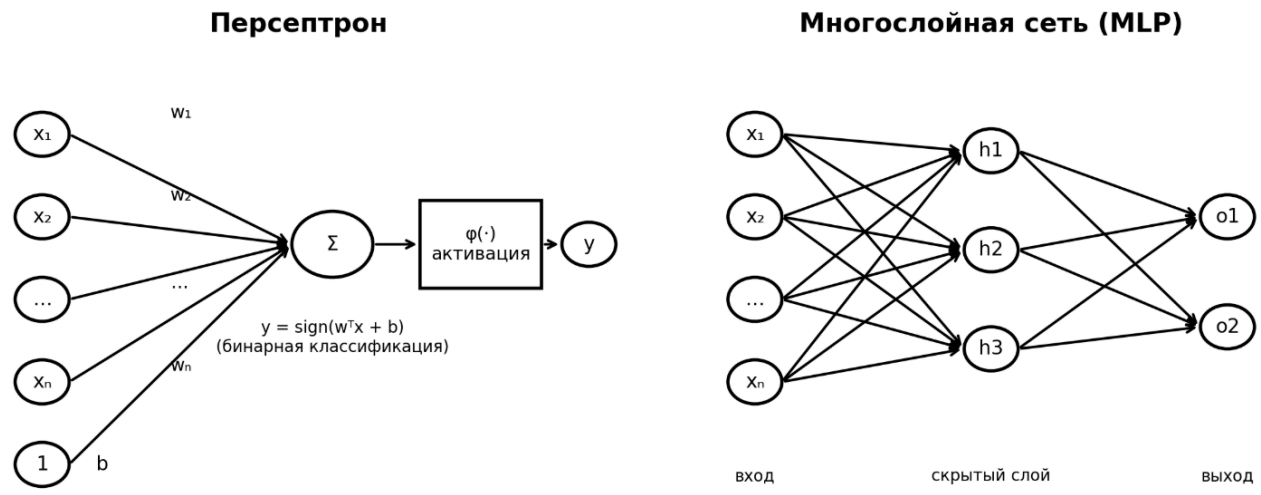
$$\mathbf{w} \leftarrow \mathbf{w} + \eta\,(t - y)\,\mathbf{x}, \qquad b \leftarrow b + \eta\,(t - y),$$

где t — истинная метка, η — шаг обучения. Персептрон гарантированно сходится, если данные **линейно разделимы**, но не решает нелинейно разделимые задачи (классический пример — XOR).

Многослойная сеть прямого распространения (MLP) — это нейросеть из нескольких слоёв (входной, один или несколько скрытых и выходной), где сигнал идёт **только вперёд**, без обратных связей. Каждый слой выполняет аффинное преобразование и нелинейность:

$$\mathbf{h}^{(l)} = \phi\Big(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\Big),$$

где ϕ — функция активации (ReLU, sigmoid, tanh), $\mathbf{h}^{(0)} = \mathbf{x}$. Наличие скрытых слоёв и нелинейных активаций позволяет моделировать **нелинейные** зависимости; MLP является универсальным аппроксиматором функций при достаточном числе нейронов.



Без формул

Виды персептронов

1) Однослойный персептрон (классический, Розенблатта)

Один слой весов, выход обычно пороговый:

$$y = \mathrm{sign}(w^T x + b)$$

Решает только **линейно разделимые** задачи (граница — гиперплоскость).

2) Многовыходный персептрон (multiclass / one-vs-rest)

Несколько выходных нейронов (по классу). Выбирают класс с максимальным откликом:

$$\hat{c} = \arg\max_k (w_k^T x + b_k)$$

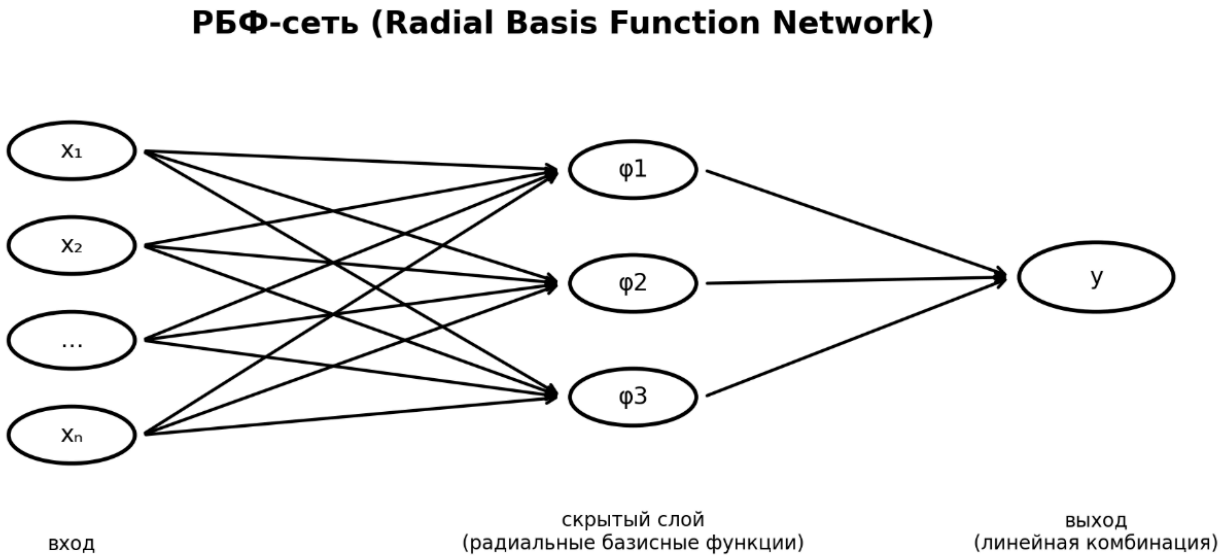
По сути набор линейных классификаторов.

3) Многослойный персептрон (MLP)

Персептрон с одним или несколькими скрытыми слоями и нелинейными активациями:

$$h = \varphi(Wx + b), \quad y = \psi(Vh + c)$$

62. Искусственные нейронные сети: РБФ-сеть



РБФ-сеть (Radial Basis Function Network) — это сеть прямого распространения из трёх частей: входной слой → скрытый слой радиальных базисных функций → линейный выход. Скрытые нейроны вычисляют “похожесть” входного вектора x на свои центры c_i, обычно гауссовой функцией

$$\varphi_i(x) = \exp\left(-\frac{\|x - c_i\|^2}{2\sigma_i^2}\right),$$

а выход формируется как линейная комбинация этих откликов

$$y = \sum_i w_i \, \varphi_i(x) + b$$

Обучение обычно двухэтапное: сначала выбирают центры c_i (например, k-means или выбор из обучающих точек) и ширины σ_i, затем веса w_i на выходе находят как задачу линейной регрессии/МНК (быстро, т.к. выход линейный). РБФ-сети хорошо аппроксимируют нелинейные зависимости за счёт локальных “колоколов” в признаковом пространстве.

---

63. Глубокие нейронные сети

Глубокие нейронные сети (Deep Neural Networks, DNN) — это нейросети с **многими слоями** (больше одного скрытого), которые учатся строить **иерархию признаков**: нижние слои выделяют простые структуры, верхние — более сложные и абстрактные. По сути это композиция преобразований вида

$$h^{(l)} = \phi\left(W^{(l)}h^{(l-1)} + b^{(l)}\right),$$

где нелинейность ϕ позволяет моделировать сложные зависимости.

Ключевые типы глубоких сетей, которые чаще всего встречаются в обработке изображений и сигналов: **CNN** (сверточные сети) для изображений, **RNN/LSTM/GRU** для последовательностей, **Transformer** (в т.ч. ViT) для текста и изображений, а также **автоэнкодеры** и **GAN** для представления и генерации данных. В компьютерном зрении CNN используют свёртки и пуллинг для выделения признаков и устойчивости к сдвигам, а современные модели часто сочетают свёртки и attention.

Обучаются глубокие сети обычно методом **обратного распространения ошибки** и градиентными оптимизаторами (SGD, Adam) по функции потерь (кросс-энтропия для классификации, MSE/MAE для регрессии). Чтобы обучение было стабильным и точным, используют техники: нормализацию (BatchNorm/LayerNorm), функции активации типа ReLU, регуляризацию (dropout, weight decay), а также data augmentation. На практике глубокие сети требуют много данных и вычислений, но дают высокое качество в распознавании, детекции, сегментации, генерации и восстановлении изображений.

---

64. Сверточные нейронные сети

Сверточные нейронные сети (CNN) — класс глубоких нейросетей, специально разработанный для обработки изображений и других данных с “решётчатой” структурой. Их ключевая идея — заменять полносвязные слои на **свёртки**, то есть применять небольшие обучаемые фильтры (ядра) ко всем участкам изображения, что резко уменьшает число параметров и делает сеть устойчивой к сдвигам объектов.

Основная операция слоя — свёртка (на практике чаще кросс-корреляция): для входной карты признаков x и ядра w

$$y_{i,j} = \sum_u \sum_v w_{u,v}\, x_{i+u,\, j+v} + b,$$

после чего обычно применяют нелинейность (например ReLU) и, при необходимости, нормализацию. В CNN используются три важных принципа: локальные рецептивные поля (нейрон “видит” только маленькое окно), **разделение весов** (одно и то же ядро применяется по всему изображению) и формирование **карт признаков** (feature maps), где каждый канал учится выделять свой тип структуры.

Для уменьшения пространственного размера и повышения устойчивости к небольшим сдвигам применяют **пулинг** (max/avg pooling) или свёртки со stride; в современных архитектурах часто используют именно stride-свёртки вместо классического пулинга. По мере углубления сети слои переходят от простых признаков (градиенты, края) к более сложным (текстуры, части объектов, целые объекты). Обучение CNN выполняется обратным распространением ошибки и градиентными оптимизаторами, как и для других DNN.

CNN применяются в задачах классификации (что на изображении), детекции (где объект), сегментации (маска объекта), восстановления изображений (денойзинг, супер-разрешение) и

65. Функции активации, используемые в глубоких нейронных сетях

1. Сигмоида (Sigmoid)

Функция активации сигмоида, или логистическая функция, является одной из самых ранних функций активации. Она преобразует входное значение в диапазон от 0 до 1:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Преимущества: сигмоида работает хорошо для задач бинарной классификации, где нужно предсказать вероятность.
Недостатки: при использовании в глубоких сетях часто возникает **проблема исчезающих градиентов** (vanishing gradients), что замедляет обучение. Также функция не центрирована вокруг нуля, что может замедлить процесс оптимизации.

---

2. Гиперболический тангенс (Tanh)

Гиперболический тангенс похож на сигмоиду, но его диапазон — от -1 до 1:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Преимущества: функция центрирована на нуле, что помогает ускорить обучение, так как данные, проходящие через неё, имеют баланс положительных и отрицательных значений.
Недостатки: также страдает от проблемы исчезающих градиентов при слишком больших или маленьких входных значениях.

---

3. ReLU (Rectified Linear Unit)

ReLU — одна из самых популярных функций активации в современных нейронных сетях. Она определена как:

$$\mathrm{ReLU}(x) = \max(0, x)$$

Преимущества: ReLU значительно ускоряет обучение, так как она даёт положительные значения для всех положительных входов, и её градиент не исчезает, как у сигмоиды и tanh. Также она вычисляется очень быстро.
Недостатки: может возникнуть **проблема "мертвых нейронов"** (dead neurons), когда нейроны никогда не активируются, если вход всегда отрицателен (из-за чего их градиент будет равен нулю).

---

66. ROC-кривая

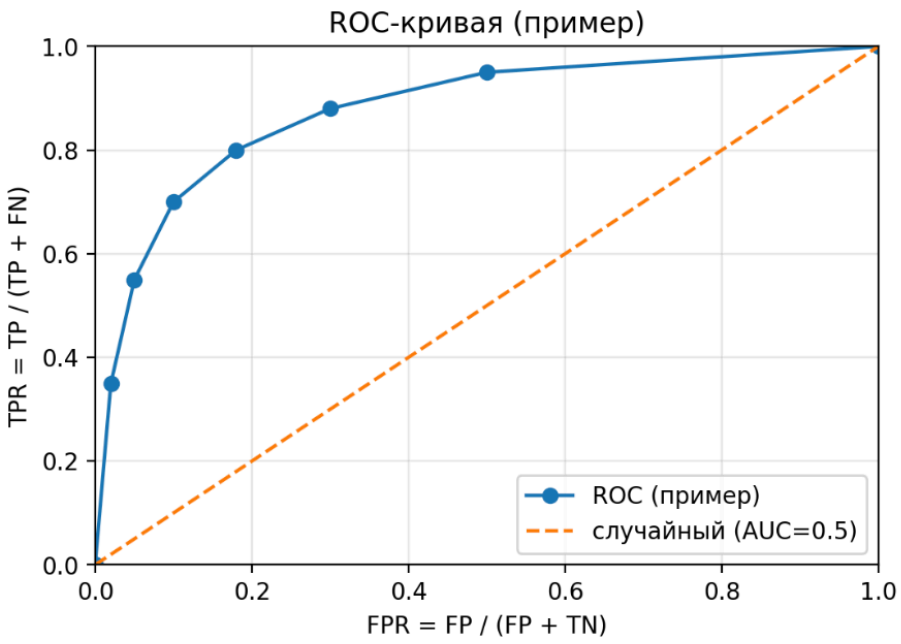
ROC-кривая (Receiver Operating Characteristic) — это график, который показывает качество **бинарного классификатора** при разных порогах решения. По оси X откладывают **FPR** (False Positive Rate, доля ложноположительных), по оси Y — **TPR** (True Positive Rate, чувствительность/recall).

Определения через матрицу ошибок:

$$TPR = \frac{TP}{TP + FN}, \qquad FPR = \frac{FP}{FP + TN}.$$

Как строят: модель выдаёт скор/вероятность принадлежности к классу “1”, затем меняют порог t. Для каждого порога считают $TPR(t)$ и $FPR(t)$ и получают кривую. Чем ближе ROC к левому верхнему углу, тем лучше: это означает высокий TPR при низком FPR. Диагональ $TPR = FPR$ соответствует случайному угадыванию.

ROC-Кривая (Пример)



---

67. Метрики для оценки точности сегментации изображений

Суть оценки сегментации — сравнить предсказанную маску с истинной по пикселям и измерить, насколько они совпадают. Самые основные метрики: **IoU (Jaccard)** $= \frac{TP}{TP+FP+FN}$ и **Dice (F1)** $= \frac{2TP}{2TP+FP+FN}$, они показывают степень перекрытия масок. Дополнительно используют **pixel accuracy** $= \frac{TP+TN}{TP+FP+FN+TN}$ и **precision/recall** (лишние пиксели vs пропуски). Если важно качество именно контуров, применяют метрики по границам, например расстояние Хаусдорфа или среднее расстояние между границами; в многоклассовой сегментации обычно берут среднее по классам (mIoU, mean Dice).

---

4) Precision / Recall

$$Precision = \frac{TP}{TP + FP}, \qquad Recall = \frac{TP}{TP + FN}$$

Precision — насколько “чистая” маска (мало лишнего), Recall — насколько “полная” (мало пропусков).

5) Specificity (True Negative Rate)

$$TNR = \frac{TN}{TN + FP}$$

Полезно, когда важно не “захватывать” фон.

---

68. Метрики для оценки точности классификации

Метрики точности классификации обычно считаются по **матрице ошибок** (confusion matrix): TP, FP, FN, TN. Суть: измерить, сколько объектов распознано правильно и какие ошибки преобладают.

Для **бинарной** классификации основные метрики такие: **Accuracy** $\frac{TP+TN}{TP+FP+FN+TN}$ показывает общую долю правильных ответов, **Precision** $\frac{TP}{TP+FP}$ показывает, насколько “чистые” положительные предсказания, **Recall/TPR** $\frac{TP}{TP+FN}$ показывает, сколько истинных положительных найдено, **F1-score** $\frac{2PR}{P+R}$ балансирует precision и recall. Часто дополнительно используют **Specificity/TNR** $\frac{TN}{TN+FP}$ и **ROC-AUC**, которая оценивает качество ранжирования по всем порогам.

Для **многоклассовой** классификации применяют средние варианты precision/recall/F1: **macro** (простое среднее по классам), **weighted** (с весами по числу объектов в классе) и **micro** (считается по суммарным TP/FP/FN). Если классы несбалансированы, accuracy может быть обманчивой, поэтому чаще опираются на F1 и AUC/PR-AUC. ↓

69. Точность, полнота, F-мера

Точность (Precision) показывает, какая доля объектов, которые модель пометила как “положительные”, действительно положительные:

$$Precision = \frac{TP}{TP + FP}.$$

Полнота (Recall, Sensitivity, TPR) показывает, какая доля всех настоящих положительных объектов была найдена моделью:

$$Recall = \frac{TP}{TP + FN}.$$

F-мера (чаще F1) — это гармоническое среднее точности и полноты, то есть один показатель, который балансирует обе метрики:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN}.$$

Интуитивно: precision штрафует за лишние срабатывания (FP), recall — за пропуски (FN), F1 учитывает оба типа ошибок.

70. Анализ движения: вычитание изображений, вычисление векторов перемещений, вычисление траекторий движущихся точек

Вычитание изображений — базовый способ обнаружить движение: берут два кадра $I_t(x, y)$ и $I_{t-1}(x, y)$ и считают разность $|I_t - I_{t-1}|$. Затем применяют порог, получая маску движения $M(x, y) = 1$, если $|I_t - I_{t-1}| > T$, иначе 0. На практике часто используют фон/модель фона и вычитают не предыдущий кадр, а оценку фона, чтобы устойчиво выделять движущиеся объекты при длительном наблюдении.

Вычисление векторов перемещений означает оценку смещения объектов или блоков между кадрами. Типичный подход — блочное сопоставление: изображение разбивают на блоки, для каждого блока ищут наиболее похожий блок в окрестности предыдущего кадра (по SSD/SAD), и разность координат даёт вектор движения (u, v). Более “плотный” подход — оптический поток, где для каждого пикселя оценивают поле скоростей $(u(x, y), v(x, y))$ на основе предположения о сохранении яркости и гладкости движения.

Вычисление траекторий движущихся точек — это трекинг: выбирают интересующие точки (углы, особенности), находят их соответствия на следующих кадрах и соединяют положения во времени, получая последовательность (x_t, y_t). Обычно сначала детектируют устойчивые точки (например Harris, Shi–Tomasi, FAST), затем отслеживают их с помощью оптического потока Лукаса–Канаде или сопоставления дескрипторов, а для сглаживания и предсказания движения используют фильтр Калмана или другие модели. Траектория получается как набор позиций точки по кадрам, а также производные величины: скорость, ускорение и направление.

---

71. Понятие «движение» в компьютерном зрении. Вектор перемещения

В компьютерном зрении под **движением** понимают изменение положения точек или объектов сцены во времени, наблюдаемое в видеопоследовательности. Формально это проявляется как изменение координат соответствующих точек изображения между кадрами t и $t + \Delta t$. Причинами видимого движения могут быть реальное перемещение объектов, движение камеры, а также изменения глубины и перспективы; поэтому в общем случае движение в изображении описывают как поле соответствий между кадрами.

Вектор перемещения — это количественное описание такого изменения положения. Если точка имела координаты (x, y) в кадре t, а в следующем кадре стала (x', y'), то её вектор перемещения задаётся как

$$\mathbf{d} = (u, v) = (x' - x,\ y' - y).$$

В задачах блокового сопоставления вектор перемещения вычисляют для блока пикселей (блок смещается на (u, v)), а в задачах оптического потока получают поле векторов $\mathbf{d}(x, y)$ для большинства пикселей. Вектор перемещения отражает направление и величину смещения на интервале между кадрами и используется для компенсации движения, трекинга и анализа траекторий.

---

72. Вычисление траекторий движущихся точек

Вычисление траекторий движущихся точек — это задача трекинга, где нужно получить последовательность положений одной и той же точки на кадрах видеоряда. Траектория точки обычно записывается как $\{(x_t, y_t)\}_{t=1}^{T}$, где (x_t, y_t) — координаты точки в кадре t. На практике процесс состоит из трёх шагов: сначала выбирают “хорошие” точки для отслеживания (углы/особенности), затем находят их соответствия на следующих кадрах, и после этого связывают найденные положения во времени, при необходимости фильтруя шум и восстанавливая пропуски

Выбор точек делают детекторами особенностей, например Harris или Shi–Tomasi, чтобы точки имели выраженную локальную структуру и не “терялись” при небольших изменениях. Соответствие точки между соседними кадрами чаще всего ищут либо по оптическому потоку (классический вариант — Lucas–Kanade), либо по сопоставлению дескрипторов (SIFT/SURF/ORB) при больших перемещениях. В случае оптического потока считается локальный вектор смещения (u_t, v_t) и координаты обновляются как $x_{t+1} = x_t + u_t$, $y_{t+1} = y_t + v_t$. Чтобы траектория была устойчивой, обычно применяют фильтрацию и предсказание, например фильтр Калмана: он сглаживает измерения, даёт прогноз следующего положения и помогает переживать кратковременные пропуски. Дополнительно используют проверку “вперёд–назад” и пороги на ошибку, чтобы отбрасывать неверные соответствия, а при потере точки выполняют переинициализацию или удаляют её из набора.

---

73. Обнаружение значительных изменений условий съемки

Обнаружение значительных изменений условий съёмки в видео — это задача выявить моменты, когда кадры резко меняются не из-за движения объектов, а из-за **смены освещения/экспозиции/баланса белого/камерных параметров** или из-за **смены сцены**. На практике это делают по статистическим признакам кадра и их скачкам во времени.

Чаще всего используют такие признаки и проверки. Сначала для каждого кадра считают глобальные характеристики яркости: среднее μ_t, дисперсию σ_t^2, иногда медиану и доли пересвета/провала в тенях. Существенное изменение условий освещения проявляется как резкий скачок $|\mu_t - \mu_{t-1}|$ или $|\sigma_t - \sigma_{t-1}|$ выше порога. Более устойчивый способ — сравнение **гистограмм** яркости (или Y в YCbCr): считают расстояние между гистограммами соседних кадров, например χ^2, пересечение гистограмм или KL-дивергенцию; большой разрыв означает смену освещения или сцены. Для изменений цвета (баланс белого) аналогично сравнивают гистограммы цветности или средние значения каналов R, G, B и их соотношения.

74. Сегментация видеопоследовательности

Сегментация видеопоследовательности — это выделение объектов/областей в каждом кадре с учётом **временной связи** между кадрами, то есть не просто “сегментация кадра”, а построение согласованных во времени масок $M_t(x, y)$ или меток $L_t(x, y)$. Главная цель — получить устойчивые контуры объектов при движении, изменении освещения и шуме, а также избежать “мигания” маски от кадра к кадру.

На практике подходы делят на несколько основных типов. Самый базовый — **вычитание фона**: строят модель фона $B(x, y)$ (например, усреднение, медиана, Mixture of Gaussians), затем для каждого кадра вычисляют разность $|I_t - B|$ и порогуют, получая маску движущихся областей; дальше применяют морфологию и связные компоненты для очистки. Этот метод хорош для стационарной камеры и выделения движущихся объектов. Если камера движется или сцена сложная, используют **оптический поток**: вычисляют поле скоростей (u, v) и сегментируют по согласованному движению, объединяя пиксели с похожими векторами в области. Ещё один класс — **сегментация с трекингом**: сначала сегментируют объект в первом кадре (вручную или автоматически), затем переносят маску на следующий кадр по вектору движения/потоку и уточняют границы; так получают стабильную маску. В современных задачах широко применяют **нейросетевые методы** (video object segmentation), которые используют 3D-свёртки или attention и учитывают временной контекст, выдавая маски сразу согласованные во времени.

---

75. Форматы представления видео: MKV, MP4, AVI

MP4 (ISO Base Media File Format) — самый распространённый контейнер для онлайн-видео и мобильных устройств. Обычно хорошо поддерживает H.264/H.265, AAC, субтитры и метаданные, подходит для стриминга и прогрессивной загрузки. Благодаря широкой совместимости MP4 часто считается “дефолтным” форматом для публикации.

MKV (Matroska) — гибкий контейнер, который поддерживает много дорожек видео и аудио, субтитры, главы, вложения и различные кодеки. Часто используется для хранения фильмов и сериалов, потому что удобно держать несколько озвучек и субтитров в одном файле. По совместимости с устройствами может уступать MP4, но по возможностям контейнера обычно шире.

AVI (Audio Video Interleave) — более старый контейнер (Microsoft), который исторически был распространён в Windows. Он менее удобен для современных возможностей: хуже поддерживает продвинутые метаданные, субтитры и некоторые современные схемы стриминга, поэтому сейчас используется реже, хотя остаётся совместимым с большим количеством плееров и может встречаться в старых архивах.

---

76. Форматы представления цифровых изображений: JPEG

JPEG (Joint Photographic Experts Group) — один из самых распространённых форматов представления **растровых** изображений, предназначенный в первую очередь для **фотографий**. Его ключевая особенность — **сжатие с потерями**, поэтому размер файла уменьшается значительно, но часть информации теряется необратимо; степень потерь задаётся параметром качества, и при повторных сохранениях ухудшение может накапливаться. Обычно JPEG хранится в файлах с расширением .jpg/.jpeg (часто в контейнерах JFIF или Exif) и поддерживает полноцветные изображения, как правило 8 бит на канал (24-битный цвет). Формат эффективен для сцен с плавными тональными переходами, но хуже подходит для текста, схем и графики с резкими границами из-за характерных артефактов сильного сжатия, таких как блочность, ringing и потеря мелких деталей.

77. Форматы представления цифровых изображений: TIFF, BMP, PNG

TIFF, BMP и PNG — это форматы хранения растровых изображений, но они ориентированы на разные задачи и отличаются способом сжатия, поддержкой прозрачности, глубиной цвета и наличием метаданных.

BMP (Bitmap) — простой формат, исторически связанный с Windows. Обычно хранит изображение практически “как есть” (часто без сжатия или с простым RLE), поэтому файлы получаются большими, зато очень легко читаются и обрабатываются. BMP используют там, где важна простота и совместимость, но для передачи и хранения больших коллекций он неэффективен.

PNG (Portable Network Graphics) — формат **без потерь**, созданный как замена GIF. Он использует эффективное сжатие (DEFLATE) и хорошо подходит для графики, интерфейсов, схем, изображений с резкими границами и текстом, потому что не вносит потерь и не даёт “JPEG-артефактов”. PNG поддерживает прозрачность (альфа-канал), разные глубины цвета и часто применяется в вебе для логотипов, скриншотов и графики.

TIFF (Tagged Image File Format) — гибкий “профессиональный” контейнер для изображений, широко используемый в полиграфии, сканировании, архивировании и научной/медицинской визуализации. TIFF может хранить данные как без сжатия, так и со сжатием без потерь (например LZW) или даже с потерями (JPEG-внутри TIFF), поддерживает высокую битность, многоканальные изображения и богатые метаданные (теги). Из-за гибкости TIFF часто выбирают для хранения “исходников” и промежуточных результатов обработки.
